Windows 系统 RGB-D 相机 SDK 开发指南（Python）

在法律允许的最大范围内，本文档是“按照现状”提供，可能存在瑕疵或错误。本公司不对本文档提供任何形式的明示或默示保证，包括但不限于适销性、质量满意度、适合特定目的、不侵犯第三方权利等保证；亦不对使用或是分发本文档导致的任何特殊、附带、偶然或间接的损害进行赔偿，包括但不限于商业利润损失、系统故障、数据或文档丢失产生的损失。

目　录

第 1 章 SDK 更新记录 1
第 2 章 基本介绍 3
2.1 概述 3
2.2 运行环境 3
2.3 适用产品 3
第 3 章 编程引导 4
3.1 主动轮询取图流程 4
3.2 回调取图流程 5
第 4 章 API 参考 1
4.1 头文件介绍 1
4.2 接口函数 1
4.2.1 MV3D_RGBD_GetSDKVersion 1
4.2.2 MV3D_RGBD_Initialize 1
4.2.3 MV3D_RGBD_Release 2
4.2.4 MV3D_RGBD_GetDeviceNumber 2
4.2.5 MV3D_RGBD_GetDeviceList 2
4.2.6 MV3D_RGBD_OpenDevice 3
4.2.7 MV3D_RGBD_OpenDeviceByName 4
4.2.8 MV3D_RGBD_OpenDeviceBySerialNumber 4
4.2.9 MV3D_RGBD_OpenDeviceByIp 5
4.2.10 MV3D_RGBD_CloseDevice 5
4.2.11 MV3D_RGBD_GetDeviceInfo 5
4.2.12 MV3D_RGBD_SetIpConfig 6
4.2.13 MV3D_RGBD_RegisterFrameCallBack 6
4.2.14 MV3D_RGBD_RegisterExceptionCallBack 7
4.2.15 MV3D_RGBD_GetStreamCfgList 8
4.2.16 MV3D_RGBD_Start 8
4.2.17 MV3D_RGBD_Stop 8
4.2.18 MV3D_RGBD_FetchFrame 9
4.2.19 MV3D_RGBD_SoftTrigger 9

4.2.20 MV3D_RGBD_SoftTriggerEx 10
4.2.21 MV3D_RGBD_Execute 10
4.2.22 MV3D_RGBD_GetCalibInfo 11
4.2.23 MV3D_RGBD_GetCameraParam 11
4.2.24 MV3D_RGBD_LocalUpgrade 12
4.2.25 MV3D_RGBD_GetUpgradeProcess 12
4.2.26 MV3D_RGBD_GetParam 13
4.2.27 MV3D_RGBD_SetParam 13
4.2.28 MV3D_RGBD_ExportAllParam 14
4.2.29 MV3D_RGBD_ImportAllParam 14
4.2.30 MV3D_RGBD_FileAccessRead 15
4.2.31 MV3D_RGBD_FileAccessWrite 15
4.2.32 MV3D_RGBD_GetFileAccessProgress 16
4.2.33 MV3D_RGBD_CheckCameraFuncSupport 16
4.2.34 MV3D_RGBD_ClearDataBuffer 17
4.2.35 MV3D_RGBD_MapDepthToPointCloud 17
4.2.36 MV3D_RGBD_MapDepthToPointCloudEx 18
4.2.37 MV3D_RGBD_ImageCoordinateTrans 18
4.2.38 MV3D_RGBD_SaveImage 19
4.2.39 MV3D_RGBD_SavePointCloudImage 20
4.2.40 MV3D_RGBD_SaveTexturedPointCloudImage 21
4.2.41 MV3D_RGBD_DisplayImage 21
4.2.42 MV3D_RGBD_MapDepthToColor 22
4.2.43 MV3D_RGBD_MapPointCloudToUV 23
4.3 数据结构 23
4.3.1 类定义 23
4.3.2 枚举 42
4.3.3 常量定义 47
第 5 章 相机参数节点表 50
5.1 常用的相机参数 50
5.2 深度图后处理的算法参数 52
第 6 章 控制台 Demo 59
6.1 BasicDemo 59
6.2 BasicDemo_Callback 61

6.3 DepthValue 63
6.4 DumpAllParams 65
6.5 FetchFrame 69
6.6 ForceDeviceIP 71
6.7 GetPointCloudImage 73
6.8 ImageAlign 76
6.9 ImageCoordinateTrans 79
6.10 ListDevices 82
6.11 LoopDetect 85
6.12 MapDepthToColor 88
6.13 MapPointCloudToUV 91
6.14 MultiDevice 94
6.15 Point3D 96
6.16 SaveImage 98
6.17 SaveTexturedPointCloudImage 101
6.18 TriggerMode 104
6.19 GetCameraParam 106
第 7 章 第三方对接 Demo 108
7.1 FetchFrame 109
7.2 GetPointCloudImage 114
7.3 ImageRead 117
7.4 RecorderRGBD 118
7.5 SaveImage 122
第 8 章 状态码定义 125
8.1 正确码定义 125
8.2 错误码定义 125
第 9 章 常见问题 127

第1章 SDK 更新记录

更新版本	更新说明
V1.4.0	● 设备触发新增执行带触发 ID 的设备软触发接口 MV3D_RGBD_SoftTriggerEx() ● 设备信息处理新增检查相机功能支持情况接口 MV3D_RGBD_CheckCameraFuncSupport() ● 图像采集新增清除数据缓存接口 MV3D_RGBD_ClearDataBuffer() ● 图像处理新增深度图转伪彩图接口 MV3D_RGBD_MapDepthToColor()和点云转 UV 坐标接口 MV3D_RGBD_MapPointCloudToUV() ● 新增 float 格式二维点数据 MV3D_RGBD_POINT_2D_F32 类定义 ● 新增 float 格式三维点数据 MV3D_RGBD_POINT_3D_F32 类定义 ● 新增法向量点云数据 MV3D_RGBD_POINT_XYZ_NORMALS 类定义 ● 新增 RGBA 信息 MV3D_RGBD_POINT_RGBA 类定义 ● 新增 RGB 信息 MV3D_RGBD_RGB_INFO 类定义 ● 新增纹理点云数据 MV3D_RGBD_POINT_XYZ_RGB 类定义 ● 新增带法向量的纹理点云数据 MV3D_RGBD_POINT_XYZ_RGB_NORMALS 类定义 ● 新增 UV 图数据 MV3D_RGBD_UV_DATA 类定义 ● 新增伪彩图转换参数 MV3D_RGBD_CONVERT_COLOR_PAPRAM 类定义 ● 新增输出点云图像类型 Mv3dRgbdPointCloudType 枚举 ● 伪彩图转换映射模式 Mv3dRgbdConvertColorMapMode 枚举 ● 新增伪彩图转换范围模式 Mv3dRgbdConvertColorRangeMode 枚举 ● 新增功能类型 Mv3dRgbdFunctionType 枚举 ● 常量定义新增输出点云图像类型 MV3D_RGBD_ENUM_POINT_CLOUD_OUTPUT ● 常量定义新增降采样参数 MV3D_RGBD_INT_DOWNSAMPLE ● 新增将网口相机日志保存到本地的功能。可通过配置文件配置，默认打开 ● 错误码新增 MV3D_RGBD_E_USB_WRITE ● 新增部分控制台 demo，主要涉及 DepthValue、DumpAllParams、FetchFrame、ForceDeviceIP 、 GetPointCloudImage 、 ImageAlign 、 ImageCoordinateTrans 、ListDevices、LoopDetect、MapDepthToColor、MapPointCloudToUV、MultiDevice、

	Point3D 、 SaveImage 、 SaveTexturedPointCloudImage 、 TriggerMode 、 GetCameraParam ● 新增部分第三方对接 demo，主要涉及 Open3D 的 FetchFrame、GetPointCloudImage、ImageRead、RecorderRGBD、SaveImage ● 新增部分常见问题 ● 更新 MV3D_RGBD_GetFileAccessProgress()接口 ● 修改 MV3D_RGBD_ImageCoordinateTrans()接口函数的介绍 ● 修改 MV3D_RGBD_SavePointCloudImage()接口函数的备注，调整支持的格式情况 ● 修改 MV3D_RGBD_SaveTexturedPointCloudImage()接口函数的备注，调整支持的格式情况 ● 修改 MV3D_RGBD_SaveImage()接口函数的介绍和备注，调整支持的格式情况 ● 修改 MV3D_RGBD_DisplayImage()接口函数的备注，调整支持的格式情况 ● 更新图像帧数据 MV3D_RGBD_FRAME_DATA 类定义，新增触发 id 参数 nTriggerId ● 更新相机深度图转 Rgb 的外参，4x4 矩阵 MV3D_RGBD_CAMERA_EXTRINSIC 类定义 ● 更新图像格式 Mv3dRgbdImageType 枚举 ● 点云图存图格式 Mv3dRgbdPointCloudFileType 枚举新增 2 种点云格式 ● 更新常量定义 MV3D_RGBD_INT_IMAGEALIGN 的说明 ● 更新部分控制台 demo，主要涉及 BasicDemo_Callback ● 更新相机参数节点表
V1.3.0	初始版本，通过 Python 接口实现 RGB-D 相机的取流、参数设置等。

第2章 基本介绍

2.1 概述

RGB-D 相机 SDK 是用于 RGB-D 相机的一个独立组件，支持图像采集、配置参数、获取设备信息等功能。

2.2 运行环境

- 硬件配置：CPU：Intel Atom (TM) E3845 及以上
  内存：4G 及以上
- 操作系统：Windows7/10 32/64 位 Windows11 64 位
  Python：3.X 版本

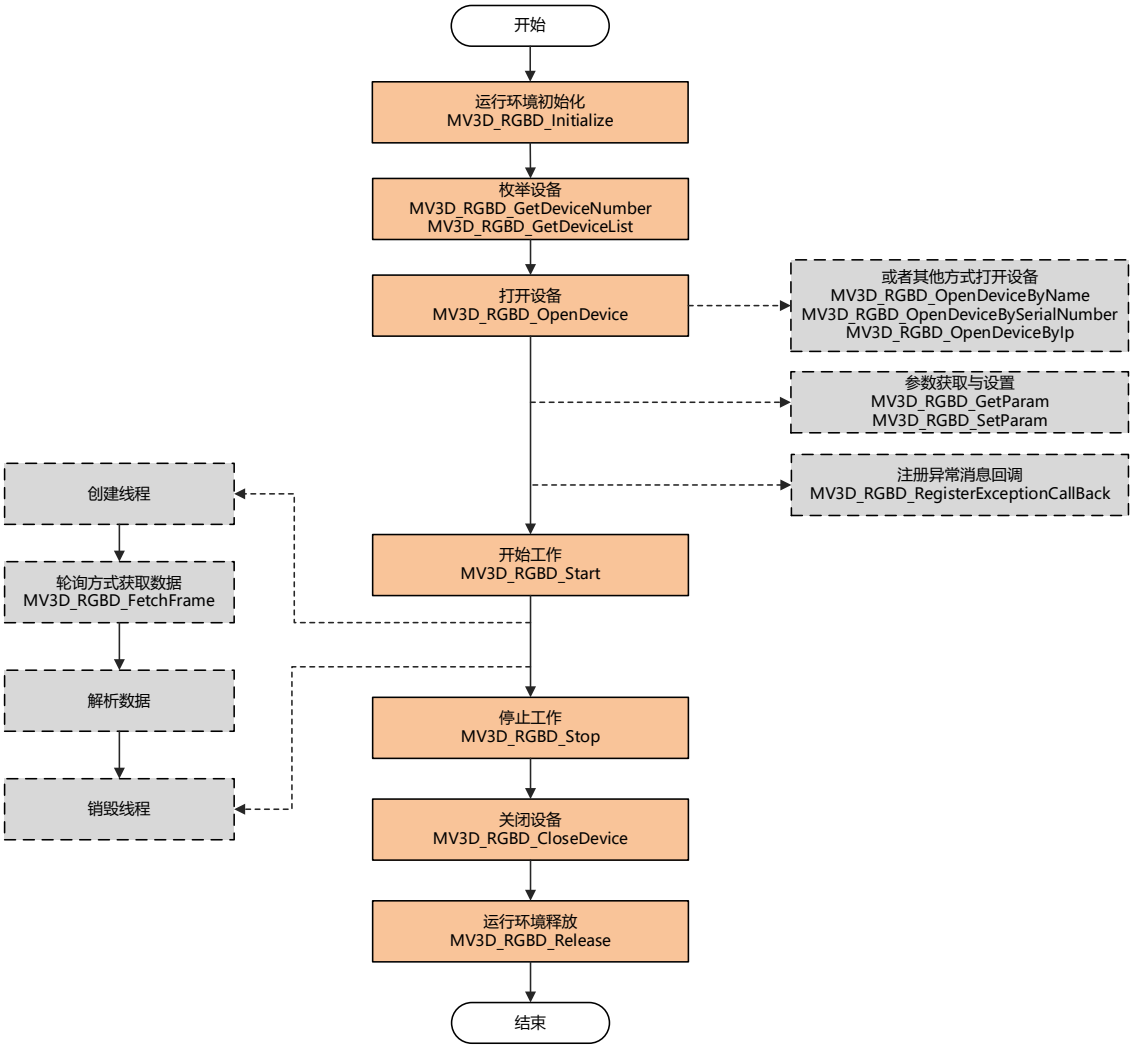
2.3 适用产品

- 激光振镜系列：激光振镜立体相机
- 散斑 RGB-D 系列：RGB-D 智能立体相机
- 感知系列：双目感知相机、ToF 感知相机

第3章 编程引导

3.1 主动轮询取图流程

本章节介绍 python 接口函数基本使用流程，具体如下图所示。



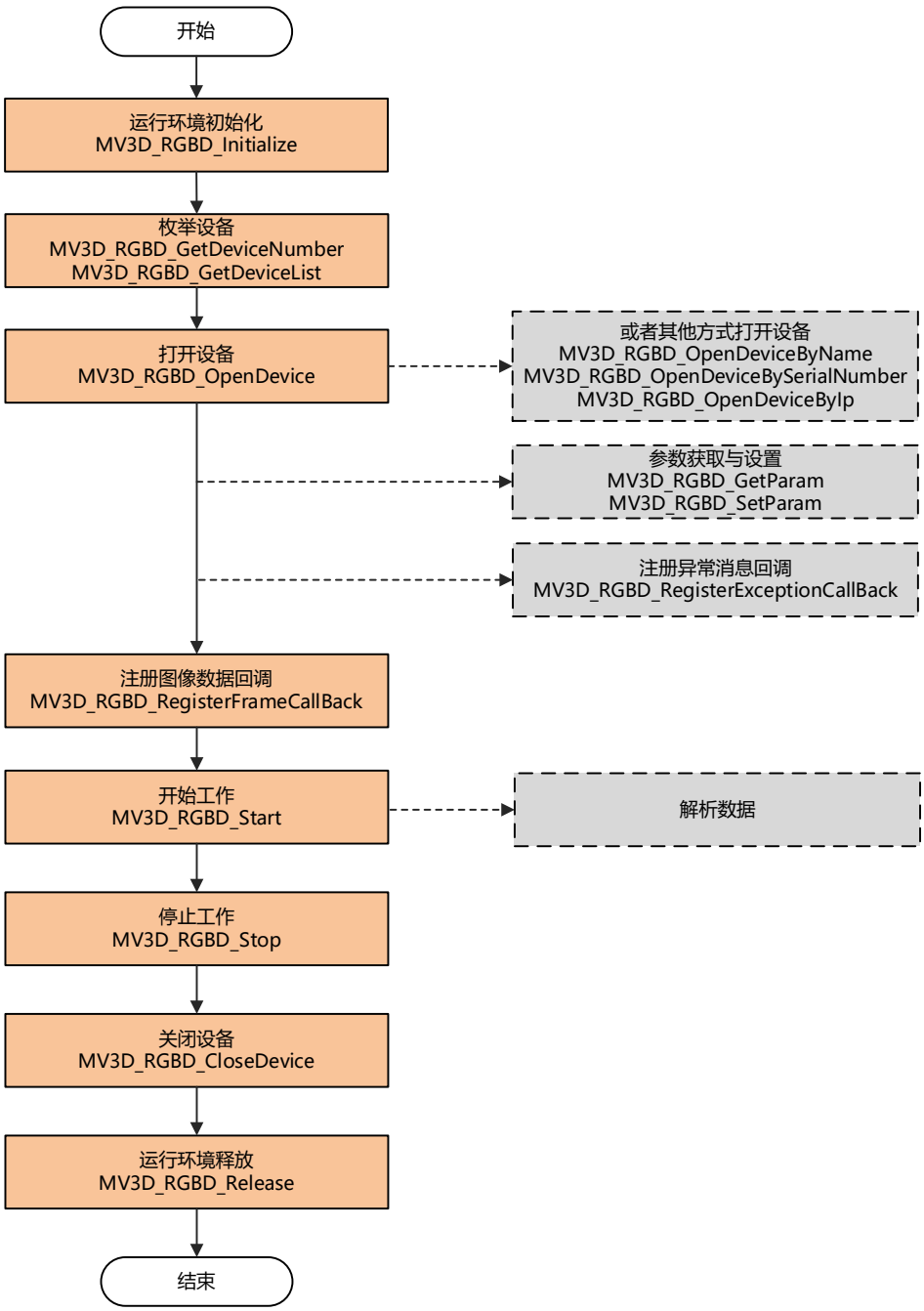
打开设备可通过以下四种方式实现。

方法	说明
MV3D_RGBD_OpenDevice()	打开设备
MV3D_RGBD_OpenDeviceByName()	通过设备型号打开设备
MV3D_RGBD_OpenDeviceBySerialNumber()	通过序列号打开设备

方法	说明
MV3D_RGBD_OpenDeviceByIp()	通过 IP 地址打开设备

3.2 回调取图流程

本章节介绍 python 接口函数基本使用流程，具体如下图所示。



打开设备可通过以下四种方式实现。

方法	说明
MV3D_RGBD_OpenDevice()	打开设备
MV3D_RGBD_OpenDeviceByName()	通过设备型号打开设备
MV3D_RGBD_OpenDeviceBySerialNumber()	通过序列号打开设备
MV3D_RGBD_OpenDeviceByIp()	通过 IP 地址打开设备

说明

使用回调函数时，应避免耗时操作、线程锁等操作。此类操作容易造成回调阻塞。

第4章 API 参考

4.1 头文件介绍

头文件名称	文件说明
Mv3dRgbdApi.py	定义接口函数
Mv3dRgbdDefine.py	定义基本数据结构和错误码

4.2 接口函数

4.2.1 MV3D_RGBD_GetSDKVersion

获取 SDK 版本号

```
MV3D_RGBD_GetSDKVersion(pstVersion):
        Mv3dRgbdDll.MV3D_RGBD_GetSDKVersion.argtype = c_void_p
        Mv3dRgbdDll.MV3D_RGBD_GetSDKVersion.restype = c_uint
        MV3D_RGBD_GetSDKVersion(MV3D_RGBD_VERSION_INFO* pstVersion);
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS
MV3D_RGBD_GetSDKVersion(MV3D_RGBD_VERSION_INFO* pstVersion);
        return Mv3dRgbdDll.MV3D_RGBD_GetSDKVersion(pstVersion)
```

参数

pstVersion [OUT] 版本信息

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.2 MV3D_RGBD_Initialize

SDK 运行环境初始化

```
MV3D_RGBD_Initialize():
        Mv3dRgbdDll.MV3D_RGBD_Initialize.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_STATUS MV3D_RGBD_Initialize();
        return Mv3dRgbdDll.MV3D_RGBD_Initialize()
```

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.3 MV3D_RGBD_Release

SDK 运行环境释放

```
MV3D_RGBD_Release():
        Mv3dRgbdDll.MV3D_RGBD_Release.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_Release();
        return Mv3dRgbdDll.MV3D_RGBD_Release()
```

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.4 MV3D_RGBD_GetDeviceNumber

获取当前环境中在线设备数量

```
MV3D_RGBD_GetDeviceNumber(nDeviceType, pDeviceNumber):
        Mv3dRgbdDll.MV3D_RGBD_GetDeviceNumber.argtype = (c_uint, c_char_p)
        Mv3dRgbdDll.MV3D_RGBD_GetDeviceNumber.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_GetDeviceNumber(uint32_t
nDeviceType, uint32_t* pDeviceNumber);
        return Mv3dRgbdDll.MV3D_RGBD_GetDeviceNumber(nDeviceType, pDeviceNumber)
```

参数

nDeviceType [IN] 设备类型，见 Mv3dRgbdDeviceType，可全部获取 (DeviceType_Ethernet | DeviceType_USB | DeviceType_Vir)

pDeviceNumber [OUT] 设备数量

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.5 MV3D_RGBD_GetDeviceList

获取设备列表

```
MV3D_RGBD_GetDeviceList(nDeviceType, pstDeviceInfos, nMaxDeviceCount, pDeviceNumber):
        Mv3dRgbdDll.MV3D_RGBD_GetDeviceList.argtype = (c_uint, c_char_p, c_uint,
c_char_p)
        Mv3dRgbdDll.MV3D_RGBD_GetDeviceList.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_GetDeviceList(uint32_t
nDeviceType, MV3D_RGBD_DEVICE_INFO* pstDeviceInfos, uint32_t nMaxDeviceCount,
uint32_t* pDeviceCount);
        return Mv3dRgbdDll.MV3D_RGBD_GetDeviceList(nDeviceType, pstDeviceInfos,
nMaxDeviceCount, pDeviceNumber)
```

参数

nDeviceType [IN] 设备类型，见 Mv3dRgbdDeviceType，可全部获取 (DeviceType_Ethernet | DeviceType_USB | DeviceType_Vir)

pstDeviceInfos [IN][OUT] 设备列表

nMaxDeviceCount [IN] 设备列表缓存最大个数

pDeviceCount [OUT] 填充列表中设备个数

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.6 MV3D_RGBD_OpenDevice

打开设备

```
MV3D_RGBD_OpenDevice(self, pstDeviceInfo):
        Mv3dRgbdDll.MV3D_RGBD_OpenDevice.argtype = (c_char_p, c_char_p)
        Mv3dRgbdDll.MV3D_RGBD_OpenDevice.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_OpenDevice(HANDLE *handle,
MV3D_RGBD_DEVICE_INFO* pstDeviceInfo = NULL);
        return Mv3dRgbdDll.MV3D_RGBD_OpenDevice(byref(self.handle), pstDeviceInfo)
```

参数

self.handle [IN][OUT] 相机句柄

pstDeviceInfo [IN] 枚举的设备信息，默认为空，打开第一个相机

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.7 MV3D_RGBD_OpenDeviceByName

通过设备型号打开设备

```
MV3D_RGBD_OpenDeviceByName(self, chDeviceName):
        Mv3dRgbdDll.MV3D_RGBD_OpenDeviceByName.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_OpenDeviceByName.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_OpenDeviceByName(HANDLE
*handle, const char* chDeviceName);
        return Mv3dRgbdDll.MV3D_RGBD_OpenDeviceByName(byref(self.handle), chDeviceName)
```

参数

self.handle　[IN][OUT]　相机句柄

chDeviceName　[IN]　设备型号

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.8 MV3D_RGBD_OpenDeviceBySerialNumber

通过序列号打开设备

```
MV3D_RGBD_OpenDeviceBySerialNumber(self, chSerialNumber):
        Mv3dRgbdDll.MV3D_RGBD_OpenDeviceBySerialNumber.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_OpenDeviceBySerialNumber.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS
MV3D_RGBD_OpenDeviceBySerialNumber(HANDLE *handle, const char* chSerialNumber);
        return Mv3dRgbdDll.MV3D_RGBD_OpenDeviceBySerialNumber(byref(self.handle),
chSerialNumber.encode('ascii'))
```

参数

self.handle　[IN][OUT]　相机句柄

chSerialNumber　[IN]　序列号

备注

设备序列号可通过 MV3D_RGBD_GetDeviceList() 函数获取。

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.9 MV3D_RGBD_OpenDeviceByIp

通过 IP 打开设备，仅网口设备有效

```
MV3D_RGBD_OpenDeviceByIp(self, chIP):
        Mv3dRgbdDll.MV3D_RGBD_OpenDeviceByIp.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_OpenDeviceByIp.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_OpenDeviceByIp(HANDLE *handle,
const char* chIP);
        return Mv3dRgbdDll.MV3D_RGBD_OpenDeviceByIp(byref(self.handle),
chIP.encode('ascii'))
```

参数

self.handle　[IN][OUT]　相机句柄

chIP　[IN]　IP 地址，例如“10.114.71.116”

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.10 MV3D_RGBD_CloseDevice

关闭设备

```
MV3D_RGBD_CloseDevice(self):
        Mv3dRgbdDll.MV3D_RGBD_CloseDevice.argtype = c_void_p
        Mv3dRgbdDll.MV3D_RGBD_CloseDevice.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_CloseDevice(HANDLE *handle);
        return Mv3dRgbdDll.MV3D_RGBD_CloseDevice(byref(self.handle))
```

参数

self.handle　[IN]　相机句柄

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.11 MV3D_RGBD_GetDeviceInfo

获取当前设备的详细信息

```
MV3D_RGBD_GetDeviceInfo(self, pstDevInfo):
        Mv3dRgbdDll.MV3D_RGBD_GetDeviceInfo.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_GetDeviceInfo.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_GetDeviceInfo(HANDLE handle,
MV3D_RGBD_DEVICE_INFO* pstDevInfo);
        return Mv3dRgbdDll.MV3D_RGBD_GetDeviceInfo(self.handle, pstDevInfo)
```

参数

self.handle　[IN]　相机句柄

pstDevInfo　[IN][OUT]　返回给调用者有关相机设备信息结构体指针

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.12 MV3D_RGBD_SetIpConfig

配置 IP，仅网口设备有效

```
MV3D_RGBD_SetIpConfig(chSerialNumber, pstIPConfig):
        Mv3dRgbdDll.MV3D_RGBD_SetIpConfig.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_SetIpConfig.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_SetIpConfig(const char*
chSerialNumber, MV3D_RGBD_IP_CONFIG* pstIPConfig);
        return Mv3dRgbdDll.MV3D_RGBD_SetIpConfig(chSerialNumber, pstIPConfig)
```

参数

chSerialNumber　[IN]　序列号

pstIPConfig　[IN]　IP 配置，静态 IP、DHCP 等

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.13 MV3D_RGBD_RegisterFrameCallBack

注册图像数据回调

```
MV3D_RGBD_RegisterFrameCallBack(self, cbOutput, pUser):
        Mv3dRgbdDll.MV3D_RGBD_RegisterFrameCallBack.argtype = (c_void_p, c_void_p,
c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_RegisterFrameCallBack.restype = c_uint
        # C原型:MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_RegisterFrameCallBack(HANDLE
handle, MV3D_RGBD_FrameDataCallBack cbOutput, void* pUser);
        return Mv3dRgbdDll.MV3D_RGBD_RegisterFrameCallBack(self.handle, cbOutput,
pUser)
```

参数

self.handle [IN] 相机句柄

cbOutput [IN] 回调函数指针

pUser [IN] 用户自定义变量

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.14 MV3D_RGBD_RegisterExceptionCallBack

注册异常消息回调

```
MV3D_RGBD_RegisterExceptionCallBack(self, cbException, pUser):
        Mv3dRgbdDll.MV3D_RGBD_RegisterExceptionCallBack.argtype = (c_void_p, c_void_p,
c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_RegisterExceptionCallBack.restype = c_uint
        # C原型:MV3D_RGBD_API MV3D_RGBD_STATUS
MV3D_RGBD_RegisterExceptionCallBack(HANDLE handle, MV3D_RGBD_ExceptionCallBack
cbException, void* pUser);
        return Mv3dRgbdDll.MV3D_RGBD_RegisterExceptionCallBack(self.handle,
cbException, pUser)
```

参数

self.handle [IN] 相机句柄

cbException [IN] 异常回调函数指针

pUser [IN] 用户自定义变量

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.15 MV3D_RGBD_GetStreamCfgList

开始取流前获取数据流配置列表

```
MV3D_RGBD_GetStreamCfgList(self, pstStreamCfgList):
        Mv3dRgbdDll.MV3D_RGBD_GetStreamCfgList.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_GetStreamCfgList.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_GetStreamCfgList(HANDLE
handle, MV3D_RGBD_STREAM_CFG_LIST* pstStreamCfgList);
        return Mv3dRgbdDll.MV3D_RGBD_GetStreamCfgList(self.handle, pstStreamCfgList)
```

参数

self.handle　[IN]　相机句柄

pstStreamCfgList　[IN]　返回给调用者数据流配置列表指针

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

备注

可通过该接口在开始取流前获取宽高信息，用于创建渲染接口。

4.2.16 MV3D_RGBD_Start

开始工作

```
MV3D_RGBD_Start(self):
        Mv3dRgbdDll.MV3D_RGBD_Start.argtype = c_void_p
        Mv3dRgbdDll.MV3D_RGBD_Start.restype = c_uint
        # C原型:MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_Start(HANDLE handle);
        return Mv3dRgbdDll.MV3D_RGBD_Start(self.handle)
```

参数

self.handle　[IN]　相机句柄

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.17 MV3D_RGBD_Stop

停止工作

```
MV3D_RGBD_Stop(self):
        Mv3dRgbdDll.MV3D_RGBD_Stop.argtype = c_void_p
        Mv3dRgbdDll.MV3D_RGBD_Stop.restype = c_uint
        # C原型:MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_Stop(HANDLE handle);
        return Mv3dRgbdDll.MV3D_RGBD_Stop(self.handle)
```

参数

self.handle　[IN]　相机句柄

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.18 MV3D_RGBD_FetchFrame

轮询方式获取帧数据

```
MV3D_RGBD_FetchFrame(self, pstFrameData, nTimeOut):
        Mv3dRgbdDll.MV3D_RGBD_FetchFrame.argtype = (c_void_p, c_void_p, c_uint)
        Mv3dRgbdDll.MV3D_RGBD_FetchFrame.restype = c_uint
        # C原型:MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_FetchFrame(HANDLE handle,
MV3D_RGBD_FRAME_DATA* pstFrameData, uint32_t nTimeOut);
        return Mv3dRgbdDll.MV3D_RGBD_FetchFrame(self.handle, pstFrameData, nTimeOut)
```

参数

self.handle　[IN]　相机句柄

pstFrameData　[IN][OUT]　数据指针

nTimeOut　[IN]　超时时间（单位：毫秒）

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.19 MV3D_RGBD_SoftTrigger

执行设备软触发

```
MV3D_RGBD_SoftTrigger(self):
        Mv3dRgbdDll.MV3D_RGBD_SoftTrigger.argtype = c_void_p
        Mv3dRgbdDll.MV3D_RGBD_SoftTrigger.restype = c_uint
        # C原型:MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_SoftTrigger(HANDLE handle);
        return Mv3dRgbdDll.MV3D_RGBD_SoftTrigger(self.handle)
```

参数

self.handle　[IN]　相机句柄

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

备注

若设备未开启触发模式时调用该接口，会开启触发模式并将触发源设置为软触发，再执行软触发命令。若设备支持触发 id 功能，软触发得到的图像包含由 SDK 层下发的触发 id。

4.2.20 MV3D_RGBD_SoftTriggerEx

执行带触发 ID 的设备软触发

```
MV3D_RGBD_SoftTriggerEx(self, nTriggerId):
        Mv3dRgbdDll.MV3D_RGBD_SoftTriggerEx.argtype = (c_void_p, c_uint)
        Mv3dRgbdDll.MV3D_RGBD_SoftTriggerEx.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_SoftTriggerEx(HANDLE handle,
uint32_t nTriggerId);
        return Mv3dRgbdDll.MV3D_RGBD_SoftTriggerEx(self.handle, nTriggerId)
```

参数

self.handle　[IN]　相机句柄

nTriggerId　[IN]　触发 ID

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

备注

若设备未开启触发模式时调用该接口，会开启触发模式并将触发源设置为软触发，再执行软触发命令。软触发得到的图像包含由软件层下发的触发 id。通过对比输入的触发 id 参数与图像帧返回的触发 id，判断当前帧是否有效。

4.2.21 MV3D_RGBD_Execute

执行设备 Command 命令

```
MV3D_RGBD_Execute(self, strKey):
        Mv3dRgbdDll.MV3D_RGBD_Execute.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_Execute.restype = c_uint
        # C原型:MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_GetCalibInfo(HANDLE handle,
uint32_t nCoordinateType, MV3D_RGBD_CALIB_INFO *pstCalibInfo);
        return Mv3dRgbdDll.MV3D_RGBD_Execute(self.handle, strKey.encode('ascii'))
```

参数

self.handle [IN] 相机句柄

strKey [IN] 属性键值

备注

相机具体参数介绍，参见相机参数节点表。

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.22 MV3D_RGBD_GetCalibInfo

获取相机当前标定信息

```
MV3D_RGBD_GetCalibInfo(self, nCoordinateType, pstCalibInfo):
        Mv3dRgbdDll.MV3D_RGBD_GetCalibInfo.argtype = (c_void_p, c_uint, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_GetCalibInfo.restype = c_uint
        # C原型:MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_GetCalibInfoEx(HANDLE handle,
MV3D_RGBD_CALIB_INFO_EX *pstCalibInfoEx);
        return Mv3dRgbdDll.MV3D_RGBD_GetCalibInfo(self.handle, nCoordinateType,
pstCalibInfo)
```

参数

self.handle [IN] 相机句柄

nCoordinateType [IN] 坐标系类型，见 Mv3dRgbdCoordinateType（坐标系类型）

pstCalibInfo [IN][OUT] 输出标定信息

备注

通过输入坐标系类型，获取对应的内参；若传“深度图坐标系”，返回深度相机（对于双目为矫正左相机）内参；若传“RGB 坐标系”，则返回 RGB 相机内参。

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.23 MV3D_RGBD_GetCameraParam

获取相机内外参信息

```
MV3D_RGBD_GetCameraParam(self, pstCameraParam):
        Mv3dRgbdDll.MV3D_RGBD_GetCameraParam.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_GetCameraParam.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_GetCameraParam(HANDLE handle,
MV3D_RGBD_CAMERA_PARAM *pstCameraParam);
        return Mv3dRgbdDll.MV3D_RGBD_GetCameraParam(self.handle, pstCameraParam)
```

参数

self.handle　[IN]　相机句柄

pstCameraParam　[IN][OUT]　输出相机内外参数信息

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.24 MV3D_RGBD_LocalUpgrade

设备升级

```
MV3D_RGBD_LocalUpgrade(self, pFilePathName):
        Mv3dRgbdDll.MV3D_RGBD_LocalUpgrade.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_LocalUpgrade.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_GetUpgradeProcess(HANDLE
handle, uint32_t* pProcess);
        return Mv3dRgbdDll.MV3D_RGBD_LocalUpgrade(self.handle,
pFilePathName.encode('ascii'))
```

参数

self.handle　[IN]　相机句柄

pFilePathName　[IN]　文件名

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.25 MV3D_RGBD_GetUpgradeProcess

获取升级进度

```
MV3D_RGBD_GetUpgradeProcess(self, pProcess):
        Mv3dRgbdDll.MV3D_RGBD_GetUpgradeProcess.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_GetUpgradeProcess.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_GetParam(HANDLE handle, const
char* strKey, MV3D_RGBD_PARAM* pstParam);
        return Mv3dRgbdDll.MV3D_RGBD_GetUpgradeProcess(self.handle, pProcess)
```

参数

self.handle [IN] 相机句柄

pProcess [OUT] 进度接收地址

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.26 MV3D_RGBD_GetParam

获取相机参数值

```
MV3D_RGBD_GetParam(self, strkey, pstParam):
        Mv3dRgbdDll.MV3D_RGBD_GetParam.argtype = (c_void_p, c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_GetParam.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_SetParam(HANDLE handle, const
char* strKey, MV3D_RGBD_PARAM* pstParam);
        return Mv3dRgbdDll.MV3D_RGBD_GetParam(self.handle, strkey.encode('ascii'),
pstParam)
```

参数

self.handle [IN] 相机句柄

strKey [IN] 属性键值

pstParam [IN] 返回的相机参数结构体指针

备注

相机具体参数介绍，参见相机参数节点表。

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.27 MV3D_RGBD_SetParam

设置相机参数值

```
MV3D_RGBD_SetParam(self, strkey, pstParam):
        Mv3dRgbdDll.MV3D_RGBD_SetParam.argtype = (c_void_p, c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_SetParam.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_ExportAllParam(HANDLE handle,
const char* pOutFileName);
        return Mv3dRgbdDll.MV3D_RGBD_SetParam(self.handle, strkey.encode('ascii'),
pstParam)
```

参数

self.handle [IN] 相机句柄

strKey [IN] 属性键值

pstParam [IN] 输入的相机参数结构体指针

备注

相机具体参数介绍，参见相机参数节点表。

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.28 MV3D_RGBD_ExportAllParam

导出相机参数

```
MV3D_RGBD_ExportAllParam(self, pOutFileName):
        Mv3dRgbdDll.MV3D_RGBD_ExportAllParam.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_ExportAllParam.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_ImportAllParam(HANDLE handle,
const char* pInFileName);
        return Mv3dRgbdDll.MV3D_RGBD_ExportAllParam(self.handle,
pOutFileName.encode('ascii'))
```

参数

self.handle [IN] 相机句柄

pOutFileName [IN] 导出文件名称

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.29 MV3D_RGBD_ImportAllParam

导入相机参数

```
MV3D_RGBD_ImportAllParam(self, pInFileName):
        Mv3dRgbdDll.MV3D_RGBD_ImportAllParam.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_ImportAllParam.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_FileAccessRead(void* handle,
MV3D_RGBD_FILE_ACCESS * pstFileAccess);
        return Mv3dRgbdDll.MV3D_RGBD_ImportAllParam(self.handle,
pInFileName.encode('ascii'))
```

参数

self.handle [IN] 相机句柄

pInFileName [IN] 导入文件名称

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.30 MV3D_RGBD_FileAccessRead

通过文件存取功能从相机内读取文件

```
MV3D_RGBD_FileAccessRead(self, pstFileAccess):
        Mv3dRgbdDll.MV3D_RGBD_FileAccessRead.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_FileAccessRead.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_FileAccessWrite(void* handle,
MV3D_RGBD_FILE_ACCESS * pstFileAccess);
        return Mv3dRgbdDll.MV3D_RGBD_FileAccessRead(self.handle, pstFileAccess)
```

参数

self.handle [IN] 相机句柄

pstFileAccess [IN] 文件存取结构体

备注

仅支持导出文件存取功能支持的文件类型，具体支持的文件类型请参见对应相机硬件用户手册中文件存取章节的介绍。

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.31 MV3D_RGBD_FileAccessWrite

通过文件存取功能将文件写入相机内

```
MV3D_RGBD_FileAccessWrite(self, pstFileAccess):
        Mv3dRgbdDll.MV3D_RGBD_FileAccessWrite.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_FileAccessWrite.restype = c_uintpstFileAccess);
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_FileAccessWrite(void* handle,
MV3D_RGBD_FILE_ACCESS * pstFileAccess);
        return Mv3dRgbdDll.MV3D_RGBD_FileAccessWrite(self.handle, pstFileAccess)
```

参数

self.handle [IN] 相机句柄

pstFileAccess [IN] 文件存取结构体

备注

仅支持导出文件存取功能支持的文件类型，具体支持的文件类型请参见对应相机硬件用户手册中文件存取章节的介绍。

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.32 MV3D_RGBD_GetFileAccessProgress

获取文件存取的进度

```
MV3D_RGBD_GetFileAccessProgress(self, pstFileAccessProgress):
        Mv3dRgbdDll.MV3D_RGBD_GetFileAccessProgress.argtype = (c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_GetFileAccessProgress.restype = c_uint
        # C: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_GetFileAccessProgress(void*
        handle, MV3D_RGBD_FILE_ACCESS_PROGRESS * pstFileAccessProgress);
        return Mv3dRgbdDll.MV3D_RGBD_GetFileAccessProgress(self.handle,
        pstFileAccessProgress)
```

参数

self.handle [IN] 相机句柄

pstFileAccessProgress [IN] 文件存取进度结构体

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.33 MV3D_RGBD_CheckCameraFuncSupport

检查相机功能支持情况

```
MV3D_RGBD_CheckCameraFuncSupport(self, enFuncType):
        Mv3dRgbdDll.MV3D_RGBD_CheckCameraFuncSupport.argtype = (c_void_p, c_uint)
        Mv3dRgbdDll.MV3D_RGBD_CheckCameraFuncSupport.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_CheckCameraFuncSupport(void*
handle, Mv3dRgbdFunctionType enFuncType);
        return Mv3dRgbdDll.MV3D_RGBD_CheckCameraFuncSupport(self.handle, enFuncType)
```

参数

self.handle [IN] 相机句柄

enFuncType [IN] 功能类型

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

备注

调用该接口检查相机是否支持某项功能，例如检查相机是否支持触发 id 功能，如果支持，则后续可以调用 MV3D_RGBD_SoftTriggerEx 接口，执行带触发 id 的软触发

4.2.34 MV3D_RGBD_ClearDataBuffer

清除数据缓存

```
def MV3D_RGBD_ClearDataBuffer(self):
    Mv3dRgbdDll.MV3D_RGBD_ClearDataBuffer.argtype = c_void_p
    Mv3dRgbdDll.MV3D_RGBD_ClearDataBuffer.restype = c_uint
    # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_ClearDataBuffer(void* handle);
    return Mv3dRgbdDll.MV3D_RGBD_ClearDataBuffer(self.handle)
```

参数

self.handle [IN] 相机句柄

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.35 MV3D_RGBD_MapDepthToPointCloud

RGBD 相机深度图像转换点云图像

```
MV3D_RGBD_MapDepthToPointCloud(self, pstDepthImage,pstPointCloudImage):
    Mv3dRgbdDll.MV3D_RGBD_MapDepthToPointCloud.argtype = (c_void_p, c_void_p,
c_void_p)
    Mv3dRgbdDll.MV3D_RGBD_MapDepthToPointCloud.restype = c_uint
    # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS  MV3D_RGBD_MapDepthToPointCloud(void*
handle, MV3D_RGBD_IMAGE_DATA* pstDepthImage, MV3D_RGBD_IMAGE_DATA*
pstPointCloudImage);
    return Mv3dRgbdDll.MV3D_RGBD_MapDepthToPointCloud(self.handle, pstDepthImage,
pstPointCloudImage)
```

参数

self.handle [IN] 相机句柄

pstDepthImage [IN] 输入深度图数据

pstPointCloudImage [OUT] 输出点云图数据

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.36 MV3D_RGBD_MapDepthToPointCloudEx

RGBD 相机深度图像转换点云图像（无句柄）

```
MV3D_RGBD_MapDepthToPointCloudEx(pstDepthImage, pstCalibInfo, fZunit,
pstPointCloudImage):
        Mv3dRgbdDll.MV3D_RGBD_MapDepthToPointCloudEx.argtype = (c_void_p, c_void_p,
c_float, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_MapDepthToPointCloudEx.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS
MV3D_RGBD_MapDepthToPointCloudEx(MV3D_RGBD_IMAGE_DATA* pstDepthImage,
MV3D_RGBD_CALIB_INFO* pstCalibInfo, float fZunit, MV3D_RGBD_IMAGE_DATA*
pstPointCloudImage);
        return Mv3dRgbdDll.MV3D_RGBD_MapDepthToPointCloudEx(pstDepthImage,
pstCalibInfo, ctypes.c_float(fZunit), pstPointCloudImage)
```

参数

pstDepthImage [IN] 深度图数据

pstCalibInfo [IN] 标定信息

fZunit [IN] 深度图量纲（mm）

pstPointCloudImage [OUT] 点云图数据

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.37 MV3D_RGBD_ImageCoordinateTrans

RGBD 相机坐标系转换

```
MV3D_RGBD_ImageCoordinateTrans(pstInImage, fZunit, pstOutImage, pstCameraParam):
        Mv3dRgbdDll.MV3D_RGBD_ImageCoordinateTrans.argtype = (c_void_p, c_float,
c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_ImageCoordinateTrans.restype = c_uint
        # C原型: MV3D_RGBD_API MV3D_RGBD_STATUS
MV3D_RGBD_ImageCoordinateTrans(MV3D_RGBD_IMAGE_DATA* pstInImage, float fZunit,
MV3D_RGBD_IMAGE_DATA* pstOutImage, MV3D_RGBD_CAMERA_PARAM* pstCameraParam);
        return Mv3dRgbdDll.MV3D_RGBD_ImageCoordinateTrans(pstInImage,
ctypes.c_float(fZunit), pstOutImage, pstCameraParam)
```

参数

pstInImage　[IN]　输入图像数据

fZunit　[IN]　深度图量纲（mm）

pstOutImage　[IN]　输出图像数据

pstCameraParam　[IN][OUT]　相机内/外参数信息

备注

在未连接相机但已获取到深度图数据，相机的内外参、分辨率以及深度图量纲时，可自行补充 MV3D_RGBD_IMAGE_DATA 和 MV3D_RGBD_CAMERA_PARAM 结构体的内容，从而将原始深度图转换为和 RGB 对齐后的深度图，或将 RGB 图像转换为对齐到深度图坐标系的 RGB 图。

连接相机时，通过 MV3D_RGBD_GetCameraParam 接口获取相机参数后，即可根据输入图像类型，将原始深度图转换为和 RGB 对齐后的深度图，或将 RGB 图像转换为对齐到深度图坐标系的 RGB 图。

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.38 MV3D_RGBD_SaveImage

深度图、RGB 图和原始图存图

```
MV3D_RGBD_SaveImage(self, pstImage, enFileType, chFileName):
        Mv3dRgbdDll.MV3D_RGBD_SaveImage.argtype = (c_void_p, c_void_p,c_int,c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_SaveImage.restype = c_uint
        # C原型:MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_SaveImage(void* handle,
MV3D_RGBD_IMAGE_DATA* pstImage, Mv3dRgbdFileType enFileType, const char* chFileName);
        return Mv3dRgbdDll.MV3D_RGBD_SaveImage(self.handle, pstImage, enFileType,
chFileName)
```

参数

self.handle [IN] 相机句柄

pstImage [IN] 图像数据

enFileType [IN] 文件类型

chFileName [IN] 文件名称

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

备注

- 深度图格式：C16
- RGB 图格式：GB8_Planar/YUV422/YUV422_NV12/YUV422_NV21
- 原始图格式：Mono8
- 支持存图格式：BMP/JPG/TIFF 格式；Mono 图存图仅支持 BMP 格式。

4.2.39 MV3D_RGBD_SavePointCloudImage

点云图存图

```
MV3D_RGBD_SavePointCloudImage(self, pstImage, enPointCloudFileType, chFileName):
        Mv3dRgbdDll.MV3D_RGBD_SavePointCloudImage.argtype = (c_void_p,
c_void_p,c_int,c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_SavePointCloudImage.restype = c_uint
        # C原型:MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_SavePointCloudImage(void*
handle, MV3D_RGBD_IMAGE_DATA* pstImage, Mv3dRgbdPointCloudFileType
enPointCloudFileType, const char* chFileName);
        return Mv3dRgbdDll.MV3D_RGBD_SavePointCloudImage(self.handle, pstImage,
enPointCloudFileType, chFileName.encode('ascii'))
```

参数

self.handle [IN] 相机句柄

pstImage [IN] 图像数据

enPointCloudFileType [IN] 点云图文件类型

chFileName [IN] 文件名称

备注

- 支持存图格式：PLY（ASCII）/CSV/OBJ/PLY（BINARY）/PCD（ASCII）/PCD（BINARY）格式。
- 纹理点云/法向量点云/带法向量的纹理点云不支持 CSV/OBJ 格式保存。

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.40 MV3D_RGBD_SaveTexturedPointCloudImage

存纹理点云图

```
MV3D_RGBD_SaveTexturedPointCloudImage(self, pstPointCloudImage, pstTexture,
enPointCloudFileType, chFileName):
        Mv3dRgbdDll.MV3D_RGBD_SaveTexturedPointCloudImage.argtype = (c_void_p,
c_void_p, c_void_p, c_int, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_SaveTexturedPointCloudImage.restype = c_uint
        # C原型：MV3D_RGBD_API MV3D_RGBD_STATUS
MV3D_RGBD_SaveTexturedPointCloudImage(void* handle, MV3D_RGBD_IMAGE_DATA*
pstPointCloudImage, MV3D_RGBD_IMAGE_DATA* pstTexture, Mv3dRgbdPointCloudFileType
enPointCloudFileType, const char* chFileName);
        return Mv3dRgbdDll.MV3D_RGBD_SaveTexturedPointCloudImage(self.handle,
pstPointCloudImage, pstTexture, enPointCloudFileType, chFileName.encode('ascii'))
```

参数

self.handle　[IN]　相机句柄

pstPointCloudImage　[IN]　点云图像数据

pstTexture　[IN]　图像纹理数据

enPointCloudFileType　[IN]　点云图文件类型

chFileName　[IN]　文件名称

备注

- 纹理图格式：RGB8_Planar/YUV422/YUV422_NV12/YUV422_NV21
- 支持存图格式：PLY（ASCII）/PLY（BINARY）/PCD（ASCII）/PCD（BINARY）格式。

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.41 MV3D_RGBD_DisplayImage

显示深度和 RGB 图像

```
MV3D_RGBD_DisplayImage(self, pstImage, hWnd):
        Mv3dRgbdDll.MV3D_RGBD_DisplayImage.argtype = (c_void_p, c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_DisplayImage.restype = c_uint
        # C原型:MV3D_RGBD_API MV3D_RGBD_STATUS MV3D_RGBD_DisplayImage(void* handle,
MV3D_RGBD_IMAGE_DATA* pstImage, void* hWnd)
        return Mv3dRgbdDll.MV3D_RGBD_DisplayImage(self.handle, pstImage, hWnd)
```

参数

self.handle [IN] 相机句柄

pstImage [IN] 图像数据

hWnd [IN] 窗口句柄

备注

- 深度图格式：16；
- RGB 图格式：RGB8_Planar/YUV422/YUV420SP_NV12/YUV420SP_NV21。

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.42 MV3D_RGBD_MapDepthToColor

将深度图转化成伪彩图

```
MV3D_RGBD_MapDepthToColor(pstDepthImage, pstConvertParam, pstColorImage):
        Mv3dRgbdDll.MV3D_RGBD_MapDepthToColor.argtype = (c_void_p, c_void_p, c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_MapDepthToColor.restype = c_uint
        # C:MV3D_RGBD_API MV3D_RGBD_STATUS
MV3D_RGBD_MapDepthToColor(MV3D_RGBD_IMAGE_DATA* pstDepthImage,
MV3D_RGBD_CONVERT_COLOR_PAPRAM* pstConvertParam, MV3D_RGBD_IMAGE_DATA* pstColorImage)
        return Mv3dRgbdDll.MV3D_RGBD_MapDepthToColor(pstDepthImage, pstConvertParam,
pstColorImage)
```

参数

pstDepthImage [IN] 深度图像数据

pstConvertParam [IN] 深度图转伪彩配置参数

pstColorImage [OUT] 伪彩图像数据

备注

- 深度图格式：16；
- RGB 图格式：RGB8_Planar。

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.2.43 MV3D_RGBD_MapPointCloudToUV

通过点云获取 UV 坐标

```
MV3D_RGBD_MapPointCloudToUV(pstCloudPointImage, pstCalibInfo, pstUvMap):
        Mv3dRgbdDll.MV3D_RGBD_MapPointCloudToUV.argtype = (c_void_p, c_void_p,
c_void_p)
        Mv3dRgbdDll.MV3D_RGBD_MapPointCloudToUV.restype = c_uint
        # C:MV3D_RGBD_API MV3D_RGBD_STATUS
MV3D_RGBD_MapPointCloudToUV(MV3D_RGBD_IMAGE_DATA* pstCloudPointImage,
MV3D_RGBD_CALIB_INFO* pstCalibInfo, MV3D_RGBD_UV_DATA* pstUvMap)
        return Mv3dRgbdDll.MV3D_RGBD_MapPointCloudToUV(pstCloudPointImage,
pstCalibInfo, pstUvMap)
```

参数

pstCloudPointImage　[IN] 点云图像数据

pstCalibInfo　[IN] 标定参数

pstUVMap [OUT]　UV 坐标数据

返回

成功，返回 MV3D_RGBD_OK；错误，返回错误码

4.3 数据结构

4.3.1 类定义

MV3D_RGBD_VERSION_INFO

版本信息

```
class _MV3D_RGBD_VERSION_INFO_(Structure):
    pass
_MV3D_RGBD_VERSION_INFO_._fields_ = [
    ('nMajor', c_uint),
    ('nMinor', c_uint),
    ('nRevision', c_uint),
]
MV3D_RGBD_VERSION_INFO = _MV3D_RGBD_VERSION_INFO_
```

成员变量

nMajor　主版本

nMinor　次版本

nRevision　修正版本

MV3D_RGBD_DEVICE_NET_INFO

网口设备信息

```
class _MV3D_RGBD_DEVICE_NET_INFO_(Structure):
    pass
Mv3dRgbdIpCfgMode = c_int

_MV3D_RGBD_DEVICE_NET_INFO_._fields_ = [
    ('chMacAddress', c_ubyte * 8),
    ('enIPCfgMode', Mv3dRgbdIpCfgMode),
    ('chCurrentIp', c_ubyte * 16),
    ('chCurrentSubNetMask', c_ubyte * 16),
    ('chDefultGateWay', c_ubyte * 16),
    ('chNetExport', c_ubyte * 16),
    ('nReserved', c_byte * 16),
]
MV3D_RGBD_DEVICE_NET_INFO = _MV3D_RGBD_DEVICE_NET_INFO_
```

成员变量

chMacAddress　Mac 地址

enIPCfgMode　当前 IP 类型

chCurrentIp　设备当前 IP

chCurrentSubNetMask　设备当前子网掩码

chDefultGateWay　设备默认网关

chNetExport　网口 IP 地址

nReserved　保留字节

MV3D_RGBD_DEVICE_USB_INFO

USB 设备信息

```
class _MV3D_RGBD_DEVICE_USB_INFO_(Structure):
    pass
Mv3dRgbdUsbProtocol = c_int

_MV3D_RGBD_DEVICE_USB_INFO_._fields_ = [
    ('nVendorId', c_uint),
    ('nProductId', c_uint),
    ('enUsbProtocol', Mv3dRgbdUsbProtocol),
    ('chDeviceGUID', c_ubyte * 64),
    ('nReserved', c_byte * 16),
]
MV3D_RGBD_DEVICE_USB_INFO = _MV3D_RGBD_DEVICE_USB_INFO_
```

成员变量

nVendorId　供应商 ID 号

nProductId　产品 ID 号

enUsbProtocol　支持的 USB 协议

chDeviceGUID　设备 GUID 号

nReserved　保留字节

MV3D_RGBD_DEVICE_INFO

设备信息

```
class _MV3D_RGBD_DEVICE_INFO_(Structure):
    pass
class MV3D_RGBD_DEVICE_SPECIAL_INFO(Union):
    pass
Mv3dRgbdDeviceType = c_int

MV3D_RGBD_DEVICE_SPECIAL_INFO._fields_ = [
    ('stNetInfo', MV3D_RGBD_DEVICE_NET_INFO),
    ('stUsbInfo', MV3D_RGBD_DEVICE_USB_INFO),
]

_MV3D_RGBD_DEVICE_INFO_._fields_ = [
    ('chManufacturerName', c_ubyte * 32),
    ('chModelName', c_ubyte * 32),
    ('chDeviceVersion', c_ubyte * 32),
    ('chManufacturerSpecificInfo', c_ubyte * 44),
    ('nDevTypeInfo', c_uint),
    ('chSerialNumber', c_ubyte * 16),
    ('chUserDefinedName', c_ubyte * 16),
    ('enDeviceType', Mv3dRgbdDeviceType),
    ('SpecialInfo', MV3D_RGBD_DEVICE_SPECIAL_INFO),
]
MV3D_RGBD_DEVICE_INFO = _MV3D_RGBD_DEVICE_INFO_
```

成员变量

SpecialInfo　不同设备特有信息

　　stNetInfo　网口设备特有

　　stUsbInfo　USB 设备特有

chManufacturerName　设备厂商

chModelName　设备型号

chDeviceVersion　设备版本

chManufacturerSpecificInfo　设备厂商特殊信息

nDevTypeInfo　设备类型信息，解析 31-24bit：产品线[0x01-标准产品；0x02-3D 产品；0x03-智能 ID 产品]

chSerialNumber　设备序列号

chUserDefinedName　设备用户自定义名称

enDeviceType　设备类型：网口、USB

MV3D_RGBD_IP_CONFIG

IP 配置

```
class _MV3D_RGBD_IP_CONFIG_(Structure):
    pass
Mv3dRgbdIpCfgMode = c_int

_MV3D_RGBD_IP_CONFIG_._fields_ = [
    ('enIPCfgMode', Mv3dRgbdIpCfgMode),
    ('chDestIp', c_ubyte * 16),
    ('chDestNetMask', c_ubyte * 16),
    ('chDestGateWay', c_ubyte * 16),
    ('nReserved', c_byte * 16),
]
MV3D_RGBD_IP_CONFIG =  MV3D_RGBD_IP_CONFIG
```

成员变量

enIPCfgMode　IP 配置模式

chDestIp　设置的目标 IP，仅静态 IP 模式下有效

chDestNetMask　设置的目标子网掩码；仅静态 IP 模式下有效

chDestGateWay　设置的目标网关；仅静态 IP 模式下有效

nReserved　保留字节

MV3D_RGBD_IMAGE_DATA

相机图像

```
class _MV3D_RGBD_IMAGE_DATA_(Structure):
    pass
Mv3dRgbdImageType = c_int # enum
Mv3dRgbdStreamType = c_int # enum
Mv3dRgbdCoordinateType = c_int # enum

_MV3D_RGBD_IMAGE_DATA_._fields_ = [
    ('enImageType', Mv3dRgbdImageType),
    ('nWidth', c_uint),
    ('nHeight', c_uint),
    ('pData', POINTER(c_ubyte)),
    ('nDataLen', c_uint),
    ('nFrameNum', c_uint),
    ('nTimeStamp', int64_t),
    ('bIsRectified', c_uint),
    ('enStreamType', Mv3dRgbdStreamType),
    ('enCoordinateType', Mv3dRgbdCoordinateType),
    ('nReserved', c_byte * 4),
]
MV3D_RGBD_IMAGE_DATA = _MV3D_RGBD_IMAGE_DATA_
```

成员变量

enImageType　图像格式

nWidth　图像宽

nHeight　图像高

pData　相机输出的图像数据

nDataLen　图像数据长度（字节）

nFrameNum　帧号，代表第几帧图像

nTimeStamp　设备上报的时间戳（设备上电从 0 开始，规则详见设备手册）

bIsRectified　是否矫正

enStreamType　流类型，用于区分图像（图像格式相同时）

enCoordinateType　坐标系类型

nReserved　保留字节

MV3D_RGBD_FRAME_DATA

图像帧数据

```
class _MV3D_RGBD_FRAME_DATA_(Structure):
    pass
Mv3dRgbdImageValid = c_int # enum

_MV3D_RGBD_FRAME_DATA_._fields_ = [
    ('nImageCount', c_uint),
    ('stImageData', MV3D_RGBD_IMAGE_DATA * MV3D_RGBD_MAX_IMAGE_COUNT),
    ('nValidInfo', c_uint)),
    ('nTriggerId', c_uint)),

    ('nReserved', c_byte * 8),
]
MV3D_RGBD_FRAME_DATA = _MV3D_RGBD_FRAME_DATA_
```

成员变量

nImageCount　图像个数，表示 stImage 数组的有效个数

stImageData　图像数组，每一个代表一种类型的图像

nValidInfo　帧有效信息：0（帧有效），1 << 0（丢包），1 << 1（触发标识符无效）

nTriggerId　触发标记

nReserved　保留字节

MV3D_RGBD_POINT_2D_F32

float 格式二维点数据

```
class _MV3D_RGBD_POINT_2D_F32_(Structure):
    pass
_MV3D_RGBD_POINT_2D_F32_._fileds_ = [
    ('fX', c_float),
    ('fY', c_float),
]
MV3D_RGBD_POINT_2D_F32 = _MV3D_RGBD_POINT_2D_F32_
```

成员变量

fX　X 轴坐标

fY　Y 轴坐标

MV3D_RGBD_POINT_3D_F32

float 格式三维点数据

```
class _MV3D_RGBD_POINT_3D_F32_(Structure):
    pass
_MV3D_RGBD_POINT_3D_F32_._fileds_ = [
    ('fX', c_float),
    ('fY', c_float),
    ('fZ', c_float),
]
MV3D_RGBD_POINT_2D_F32 = _MV3D_RGBD_POINT_2D_F32_
```

成员变量

fX　X 轴坐标

fY　Y 轴坐标

fZ　Z 轴坐标

MV3D_RGBD_POINT_XYZ_NORMALS

法向量点云数据

```
class _MV3D_RGBD_POINT_XYZ_NORMALS_(Structure):
    pass
_MV3D_RGBD_POINT_XYZ_NORMALS_._fileds_ = [
    ('stPoint3f', MV3D_RGBD_POINT_3D_F32),
    ('stNormals', MV3D_RGBD_POINT_3D_F32),
]
MV3D_RGBD_POINT_XYZ_NORMALS = _MV3D_RGBD_POINT_XYZ_NORMALS_
```

成员变量

stPoint3f　点云数据

stNormals　法向量数据

MV3D_RGBD_POINT_RGBA

RGBA 数据

```
class _MV3D_RGBD_POINT_RGBA_(Structure):
    pass
_MV3D_RGBD_POINT_RGBA_._fileds_ = [
    ('nR', c_uint8),
    ('nG', c_uint8),
    ('nB', c_uint8),
    ('nA', c_uint8),
]
MV3D_RGBD_POINT_RGBA = _MV3D_RGBD_POINT_RGBA_
```

成员变量

nR　R 通道数据

nG　G 通道数据

nB　B 通道数据

nA　A 通道数据

MV3D_RGBD_RGB_INFO

RGB 信息

```
class _MV3D_RGBD_RGB_INFO_(Union):
    pass
_MV3D_RGBD_RGB_INFO_._fileds_ = [
    ('stRgba', MV3D_RGBD_POINT_RGBA),
    ('fRgb', c_float),
]
MV3D_RGBD_RGB_INFO = _MV3D_RGBD_RGB_INFO_
```

成员变量

stRgba　RGBA 数据

fRgb　float 格式 RGB 信息

MV3D_RGBD_POINT_XYZ_RGB

纹理点云数据

```
class _MV3D_RGBD_POINT_XYZ_RGB_(Structure):
    pass
_MV3D_RGBD_POINT_XYZ_RGB_._fields_ = [
    ('stPoint3f', MV3D_RGBD_POINT_3D_F32),
    ('stRgbInfo', MV3D_RGBD_RGB_INFO),
]
MV3D_RGBD_POINT_XYZ_RGB = _MV3D_RGBD_POINT_XYZ_RGB_
```

成员变量

stPoint3f 点云数据

stRgbInfo RGB 信息

MV3D_RGBD_POINT_XYZ_RGB_NORMALS

带法向量的纹理点云数据

```
class _MV3D_RGBD_POINT_XYZ_RGB_NORMALS_(Structure):
    pass
_MV3D_RGBD_POINT_XYZ_RGB_NORMALS_._fields_ = [
    ('stRgbPoint', MV3D_RGBD_POINT_XYZ_RGB),
    ('stNormals', MV3D_RGBD_POINT_3D_F32),
]
MV3D_RGBD_POINT_XYZ_RGB_NORMALS = _MV3D_RGBD_POINT_XYZ_RGB_NORMALS_
```

成员变量

stRgbPoint 彩色点云数据

stNormals 法向量数据

MV3D_RGBD_UV_DATA

UV 图数据

```
class _MV3D_RGBD_UV_DATA_(Structure):
    pass
_MV3D_RGBD_UV_DATA_._fields_ = [
    ('nDataLen',c_uint),
    ('pData',POINTER(MV3D_RGBD_POINT_2D_F32)),
    ('nReserved', c_byte * 8),
]
MV3D_RGBD_UV_DATA = _MV3D_RGBD_UV_DATA_
```

成员变量

nDataLen　数据长度

pData　UV 图数据

nReserved　保留字节

MV3D_RGBD_CONVERT_COLOR_PAPRAM

深度图转伪彩参数

```
class _MV3D_RGBD_CONVERT_COLOR_PAPRAM_(Structure):
    pass
Mv3dRgbdConvertColorMapMode = c_int
Mv3dRgbdConvertColorRangeMode = c_int
_MV3D_RGBD_CONVERT_COLOR_PAPRAM_._fields_ = [
    ('enConvertColorMapMode',Mv3dRgbdConvertColorMapMode),
    ('enConvertColorRangeMode',Mv3dRgbdConvertColorRangeMode),
    ('bExistInvalidValue',c_bool),
    ('fInvalidValue',c_float * 8),
    ('fRangeMinValue',c_float),
    ('fRangeMaxValue',c_float),
    ('nReserved', c_byte * 8),
]
MV3D_RGBD_CONVERT_COLOR_PAPRAM = _MV3D_RGBD_CONVERT_COLOR_PAPRAM_
```

成员变量

enConvertColorMapMode 颜色映射模式

enConvertColorRangeMode　深度图转伪彩范围模式

bExistInvalidValue　无效值数量

fInvalidValue 无效值

fRangeMinValue　范围模式为用户指定绝对值时，表示最小深度值；范围模式为用户指定百分比时，表示最小百分比[0.0,1.0]

fRangeMaxValue　范围模式为用户指定绝对值时，表示最大深度值；范围模式为用户指定百分比时，表示最大百分比[0.0,1.0]

nReserved　保留字节

MV3D_RGBD_STREAM_CFG

固件输出的图像附加信息

```
class _MV3D_RGBD_STREAM_CFG_(Structure):
    pass
Mv3dRgbdImageType = c_int # enum

_MV3D_RGBD_STREAM_CFG_._fields_ = [
    ('enImageType', Mv3dRgbdImageType),
    ('nWidth', c_uint),
    ('nHeight', c_uint),

    ('nReserved', c_byte * 32),
]
MV3D_RGBD_STREAM_CFG =  MV3D_RGBD_STREAM_CFG
```

成员变量

enImageType　图像格式

nWidth　图像宽

nHeight　图像高

nReserved　保留字节

MV3D_RGBD_STREAM_CFG_LIST

固件输出的图像帧附加信息

```
class _MV3D_RGBD_STREAM_CFG_LIST_(Structure):
    pass
_MV3D_RGBD_STREAM_CFG_LIST_._fields_ = [
    ('nStreamCfgCount', c_uint),
    ('stStreamCfg', MV3D_RGBD_STREAM_CFG * MV3D_RGBD_MAX_IMAGE_COUNT),

    ('nReserved', c_byte * 16),
]
MV3D_RGBD_STREAM_CFG_LIST = _MV3D_RGBD_STREAM_CFG_LIST_
```

成员变量

nStreamCfgCount　图像信息数量

stStreamCfg　图像附加信息

nReserved　保留字节

MV3D_RGBD_DEVICE_INFO_LIST

枚举设备列表信息

```
class _MV3D_RGBD_DEVICE_INFO_LIST_(Structure):
pass
_MV3D_RGBD_DEVICE_INFO_LIST_._fields_ = [
('DeviceInfo', MV3D_RGBD_DEVICE_INFO * 20),
]
MV3D_RGBD_DEVICE_INFO_LIST = _MV3D_RGBD_DEVICE_INFO_LIST_
```

成员变量

DeviceInfo　设备信息结构体数组，目前最大 20

MV3D_RGBD_CAMERA_INTRINSIC

相机内参，3x3 矩阵

```
class _MV3D_RGBD_CAMERA_INTRINSIC_(Structure):
    pass
_MV3D_RGBD_CAMERA_INTRINSIC_._fields_ = [
    ('fData', c_float * 9),
]
MV3D_RGBD_CAMERA_INTRINSIC = _MV3D_RGBD_CAMERA_INTRINSIC_
```

成员变量

fData　相机内参：fx,0,cx,0,fy,cy,0,0,1

| fx| 0 | cx|

| 0 | fy| cy|

| 0 | 0 | 1 |

MV3D_RGBD_CAMERA_DISTORTION

相机畸变系数

```
class _MV3D_RGBD_CAMERA_DISTORTION_(Structure):
    pass
_MV3D_RGBD_CAMERA_DISTORTION_._fields_ = [
    ('fData', c_float * 12),
]
MV3D_RGBD_CAMERA_DISTORTION = _MV3D_RGBD_CAMERA_DISTORTION_
```

成员变量

fData　畸变系数 k1，k2，p1，p2，k3，k4，k5，k6，s1，s2，s3，s4

MV3D_RGBD_CALIB_INFO

相机的内参和畸变系数信息

```
class _MV3D_RGBD_CALIB_INFO_(Structure):
    pass
_MV3D_RGBD_CALIB_INFO_._fields_ = [
    ('stIntrinsic', MV3D_RGBD_CAMERA_INTRINSIC),
    ('stDistortion', MV3D_RGBD_CAMERA_DISTORTION),
    ('nReserved', c_byte * 16),
]
MV3D_RGBD_CALIB_INFO = _MV3D_RGBD_CALIB_INFO_
```

成员变量

stIntrinsic　内参参数：fx,0,cx,0,fy,cy,0,0,1

stDistortion　畸变系数：k1,k2,p1,p2,k3,k4,k5,k6,s1,s2,s3,s4

nReserved　保留字节

MV3D_RGBD_CAMERA_EXTRINSIC

相机深度图转 Rgb 的外参，4x4 矩阵

```
class _MV3D_RGBD_CAMERA_EXTRINSIC_(Structure):
    pass
_MV3D_RGBD_CAMERA_EXTRINSIC_._fields_ = [
    ('fData', c_float * 16),
]
MV3D_RGBD_CAMERA_EXTRINSIC = _MV3D_RGBD_CAMERA_EXTRINSIC_
```

成员变量

fData　深度图转 Rgb 外参参数：r00,r01,r02,t0,r10,r11,r12,t1,r20,r21,r22,t2,0,0,0,1

| r00| r01| r02| t0|

| r10| r11| r12| t1|

| r20| r21| r22| t2|

|　0|　0|　0|　1|

MV3D_RGBD_CAMERA_PARAM

相机参数信息

```
class _MV3D_RGBD_CAMERA_PARAM_(Structure):
    pass
_MV3D_RGBD_CAMERA_PARAM_._fields_ = [
    ('stDepthCalibInfo', MV3D_RGBD_CALIB_INFO),
    ('stRgbCalibInfo', MV3D_RGBD_CALIB_INFO),
    ('stDepth2RgbExtrinsic', MV3D_RGBD_CAMERA_EXTRINSIC),
]
MV3D_RGBD_CAMERA_PARAM = _MV3D_RGBD_CAMERA_PARAM_
```

成员变量

stDepthCalibInfo　左目内参和畸变矩阵信息

stRgbCalibInfo　rgb 内参和畸变矩阵信息

stDepth2RgbExtrinsic　相机深度图转 Rgb 的外参

MV3D_RGBD_INTPARAM

Int 类型值

```
class _MV3D_RGBD_INTPARAM_(Structure):
    pass
_MV3D_RGBD_INTPARAM_._fields_ = [
    ('nCurValue', int64_t),
    ('nMax', int64_t),
    ('nMin', int64_t),
    ('nInc', int64_t),
]
MV3D_RGBD_INTPARAM = _MV3D_RGBD_INTPARAM_
```

成员变量

nCurValue　当前值

nMax　最大值

nMin　最小值

nInc　增量值

MV3D_RGBD_ENUMPARAM

Enum 类型值

```
class _MV3D_RGBD_ENUMPARAM_(Structure):
    pass
_MV3D_RGBD_ENUMPARAM_._fields_ = [
    ('nCurValue', c_uint),
    ('nSupportedNum', c_uint),
    ('nSupportValue', c_uint * MV3D_RGBD_MAX_ENUM_COUNT),
]
MV3D_RGBD_ENUMPARAM = _MV3D_RGBD_ENUMPARAM_
```

成员变量

nCurValue　当前值

nSupportedNum　有效数据个数

nSupportValue　支持的枚举类型

MV3D_RGBD_FLOATPARAM

Float 类型值

```
class _MV3D_RGBD_FLOATPARAM_(Structure):
    pass
_MV3D_RGBD_FLOATPARAM_._fields_ = [
    ('fCurValue', c_float),
    ('fMax', c_float),
```

成员变量

fCurValue　当前值

fMax　最大值

fMin　最小值

MV3D_RGBD_STRINGPARAM

String 类型值

```
class _MV3D_RGBD_STRINGPARAM_(Structure):
    pass
_MV3D_RGBD_STRINGPARAM_._fields_ = [
    ('chCurValue', c_char * MV3D_RGBD_MAX_STRING_LENGTH),
    ('nMaxLength', uint32_t),
]
MV3D_RGBD_STRINGPARAM = _MV3D_RGBD_STRINGPARAM_
```

成员变量

chCurValue　当前值

nMaxLength　属性节点能设置字符的最大长度

MV3D_RGBD_PARAM

相机参数

```
class _MV3D_RGBD_PARAM_(Structure):
    pass

Mv3dRgbdParamType = c_int

class MV3D_RGBD_PARAM_ParamInfo(Union):
    pass
MV3D_RGBD_PARAM_ParamInfo._fields_=[
    ('bBoolParam', c_bool),
    ('stIntParam', MV3D_RGBD_INTPARAM),
    ('stFloatParam', MV3D_RGBD_FLOATPARAM),
    ('stEnumParam', MV3D_RGBD_ENUMPARAM),
    ('stStringParam', MV3D_RGBD_STRINGPARAM),
]

_MV3D_RGBD_PARAM_._fields_=[
    ('enParamType', Mv3dRgbdParamType),
    ('ParamInfo', MV3D_RGBD_PARAM_ParamInfo),
    ('nReserved', c_byte*16),
]
MV3D_RGBD_PARAM = _MV3D_RGBD_PARAM_
```

成员变量

enParamType　设置属性值类型

ParamInfo　数据信息

bBoolParam　BOOL 类型数据

stIntParam　INT 类型数据

stFloatParam　FLOAT 类型数据

stEnumParam　ENUM 类型数据

stStringParam　STRING 类型数据

nReserved　保留字节

MV3D_RGBD_EXCEPTION_INFO

异常信息

```
class _MV3D_RGBD_EXCEPTION_INFO_(Structure):
    pass
Mv3dRgbdDevException = c_int
_MV3D_RGBD_EXCEPTION_INFO_._fields_=[
    ('enExceptionId',Mv3dRgbdDevException),
    ('chExceptionDes',c_char*MV3D_RGBD_MAX_STRING_LENGTH),
    ('nReserved',c_byte*4),
]
MV3D_RGBD_EXCEPTION_INFO=_MV3D_RGBD_EXCEPTION_INFO_
```

成员变量

enExceptionId　异常 ID

chExceptionDes　异常描述

nReserved　保留字节

MV3D_RGBD_FILE_ACCESS

文件存取

```
class _MV3D_RGBD_FILE_ACCESS_(Structure):
    pass
_MV3D_RGBD_FILE_ACCESS_._fields_ = [
    ('pUserFileName', c_char_p),
    ('pDevFileName', c_char_p),
    ('nReserved', c_byte*32),
]
MV3D_RGBD_FILE_ACCESS = _MV3D_RGBD_FILE_ACCESS_
```

成员变量

pUserFileName　用户文件名

pDevFileName　设备文件名

nReserved　预留字节

MV3D_RGBD_FILE_ACCESS

文件存取进度

```
class _MV3D_RGBD_FILE_ACCESS_PROGRESS_(Structure):
    pass
_MV3D_RGBD_FILE_ACCESS_PROGRESS_._fields_ = [
    ('nCompleted', c_int64),
    ('nTotal', c_int64),

    ('nReserved', c_byte*32),
]
MV3D_RGBD_FILE_ACCESS_PROGRESS = _MV3D_RGBD_FILE_ACCESS_PROGRESS_
```

成员变量

nComplete　已完成的长度

nTotal　总长度

nReserved　保留字节

4.3.2 枚举

Mv3dRgbdDeviceType（设备类型）

枚举成员	枚举值	说明
DeviceType_Ethernet	0	网口设备
DeviceType_USB	1	USB 设备
DeviceType_Ethernet_Vir	2	虚拟网口设备
DeviceType_USB_Vir	3	虚拟 USB 设备

Mv3dRgbdIpCfgMode（IP 类型）

枚举成员	枚举值	说明
IpCfgMode_Static	1	静态 IP
IpCfgMode_DHCP	2	自动分配 IP(DHCP)
IpCfgMode_LLA	4	自动分配 IP(LLA)

Mv3dRgbdUsbProtocol（USB 协议）

枚举成员	枚举值	说明
UsbProtocol_USB2	1	USB2 协议
UsbProtocol_USB3	2	USB3 协议

Mv3dRgbdImageType（图像格式）

枚举成员	枚举值	说明
ImageType_Undefined	-1	未定义的图像类型
ImageType_Mono8	17301505	原始图(Mono8)
ImageType_Mono16	17825799	相位图(Mono16)
ImageType_Depth	17825976	深度图(C16)
ImageType_YUV422	34603058	彩色图(YUV422)
ImageType_YUV420SP_NV12	34373633	彩色图(YUV420SP_NV12)
ImageType_YUV420SP_NV21	34373634	彩色图(YUV420SP_NV21)
ImageType_RGB8_Planar	35127329	彩色图(RGB8_Planar)
ImageType_PointCloud	39846080	点云图(ABC32f)
ImageType_PointCloudWithNormals	-2134900735	带法向量的点云图 (MV3D_RGBD_POINT_XYZ_NORMALS)
ImageType_TexturedPointCloud	-2139619326	纹理点云图 (MV3D_RGBD_POINT_XYZ_RGB)
ImageType_TexturedPointCloud WithNormals	-2133327869	带法向量的纹理点云图 (MV3D_RGBD_POINT_XYZ_RGB_ NORMALS)
ImageType_Jpeg	-2145910783	Jpeg 格式
ImageType_Rgbd	-2111295481	rgbd 格式

Mv3dRgbdStreamType（数据流类型）

枚举成员	枚举值	说明
StreamType_Undefined	0	数据流未定义
StreamType_Depth	1	深度图数据流
StreamType_Color	2	Color 图数据流
StreamType_Ir_Left	3	矫正后的左目图数据流
StreamType_Ir_Right	4	矫正后的右目图数据流
StreamType_Imu	5	IMU 数据流
StreamType_LeftMono	6	左目泛光图数据流
StreamType_Mask	7	掩膜图数据流
StreamType_Mono	8	Mono 图未矫正的左右目融合图数据流
StreamType_Phase	9	相位图数据流
StreamType_Rgbd	10	RGB-D 图数据流

Mv3dRgbdCoordinateType（坐标系类型）

枚举成员	枚举值	说明
CoordinateType_Undefined	0	坐标系未定义
CoordinateType_Depth	1	深度图坐标系
CoordinateType_RGB	2	RGB 图坐标系

Mv3dRgbdDevException（异常信息）

枚举成员	枚举值	说明
DevException_Disconnect	1	设备断开连接

Mv3dRgbdParamType（参数类型）

枚举成员	枚举值	说明
ParamType_Bool	1	BOOL 型
ParamType_Int	2	整型
ParamType_Float	3	浮点型
ParamType_Enum	4	枚举型
ParamType_String	5	字符型

Mv3dRgbdFileType（深度图与 RGB 图存图格式）

枚举成员	枚举值	说明
FileType_BMP	1	BMP 格式
FileType_JPG	2	JPG 格式
FileType_TIFF	3	TIFF 格式

Mv3dRgbdPointCloudFileType（点云图存图格式）

枚举成员	枚举值	说明
PointCloudFileType_PLY	1	PLY 格式
PointCloudFileType_CSV	2	CSV 格式
PointCloudFileType_OBJ	3	OBJ 格式
PointCloudFileType_PLY_Binary	4	PLY（BINARY）格式
PointCloudFileType_PCD_ASCII	5	PCD（ASCII）格式
PointCloudFileType_PCD_Binary	6	PCD（BINARY）格式

Mv3dRgbdPointCloudType（输出点云图像类型）

枚举成员	枚举值	说明
PointCloudType_Undefined	0	不输出点云
PointCloudType_Common	1	点云图像
PointCloudType_Normals	2	带法向量信息的点云图像
PointCloudType_Texture	3	纹理点云图像
PointCloudType_Texture_Normals	4	带法向量的纹理点云图像

Mv3dRgbdConvertColorMapMode（伪彩图转换映射模式）

枚举成员	枚举值	说明
ConvertColorMapMode_Rainbow	1	彩色
ConvertColorMapMode_Dark_Rainbow	2	暗彩色
ConvertColorMapMode_Dark_Green	3	暗绿色
ConvertColorMapMode_Pinkish_Red	4	粉红色
ConvertColorMapMode_Yellow	5	黄色
ConvertColorMapMode_Gray_Scale	6	灰度

Mv3dRgbdConvertColorRangeMode（伪彩图转换范围模式）

枚举成员	枚举值	说明
ConvertColorRangeMode_Auto	0	自动
ConvertColorRangeMode_Abs	1	绝对值
ConvertColorRangeMode_Percentage	2	百分比

Mv3dRgbdFunctionType（功能类型）

枚举成员	枚举值	说明
FunctionType_TriggerId	1	触发 id

4.3.3 常量定义

常量定义	常量值	说明
MV3D_RGBD_MAX_IMAGE_COUNT	10	最大的图片个数
MV3D_RGBD_MAX_STRING_LENGTH	256	最大字符串长度
MV3D_RGBD_MAX_PATH_LENGTH	256	路径的最大字符长度
MV3D_RGBD_MAX_ENUM_COUNT	16	最大枚举数量

像素类型	像素值	说明
MV3D_RGBD_PIXEL_MONO	0x01000000	Mono 图像格式基础类型
MV3D_RGBD_PIXEL_COLOR	0x02000000	彩色图像格式基础类型
MV3D_RGBD_PIXEL_CUSTOM	0x80000000	自定义图像格式基础类型

设备属性 key 值相关定义	key 值	说明
MV3D_RGBD_INT_WIDTH	Width	图像宽
MV3D_RGBD_INT_HEIGHT	Height	图像高
MV3D_RGBD_ENUM_WORKINGMODE	CameraWorkingMode	工作模式
MV3D_RGBD_ENUM_PIXELFORMAT	PixelFormat	像素格式
MV3D_RGBD_ENUM_IMAGEMODE	ImageMode	图像模式
MV3D_RGBD_FLOAT_GAIN	Gain	增益，单位 dB
MV3D_RGBD_FLOAT_EXPOSURETIME	ExposureTime	曝光时间，单位 μs
MV3D_RGBD_FLOAT_FRAMERATE	AcquisitionFrameRate	采集帧率，单位 fps
MV3D_RGBD_ENUM_TRIGGERSELECTOR	TriggerSelector	触发选择器
MV3D_RGBD_ENUM_TRIGGERMODE	TriggerMode	触发模式
MV3D_RGBD_ENUM_TRIGGERSOURCE	TriggerSource	触发源
MV3D_RGBD_FLOAT_TRIGGERDELAY	TriggerDelay	触发延迟时间，单位 μs
MV3D_RGBD_INT_IMAGEALIGN	ImageAlign	对齐坐标系（默认值 1：对齐到彩色图坐标系；0：不对齐；2：对齐到深度图坐标系），重启程序后恢复默认值
MV3D_RGBD_BOOL_LASERENABLE	LaserEnable	投射器使能
Mv3D_RGBD_FLOAT_BASEDISTANCE	BaseDistance	左右目基线距
MV3D_RGBD_ENUM_RESOLUTION	BinningSelector	采集分辨率
MV3D_RGBD_INT_OUTPUT_RGBD	OutputRgbd	RGBD 图像输出（默认值 0：不输出；1：输出），重启程序后恢复默认值
MV3D_RGBD_INT_DEVICE_TIMEOUT	DeviceTimeout	设备控制超时时间，单位 ms
MV3D_RGBD_INT_IMAGE_NODE_NUM	ImageNodeNum	图像缓存节点个数
MV3D_RGBD_FLOAT_Z_UNIT	ZUnit	深度图量纲，单位 mm

MV3D_RGBD_ENUM_POINT_CLOUD_OUTPUT	PointCloudOutput	输出点云数据类型，详见 Mv3dRgbdPointCloudType
MV3D_RGBD_INT_DOWNSAMPLE	DownSample	降采样参数（默认值 0：不输出；1：图像宽高降低一半）

设备文件枚举常量	枚举常量值	说明
MV3D_RGBD_SENSOR_CALI	RGBDSensorCali	相机传感器标定文件
MV3D_RGBD_HANDEYE_CALI	RGBDHandEyeCali	相机手眼标定文件

第5章 相机参数节点表

本章节列举常用的相机参数节点以及深度图后处理的算法参数。以下参数可通过调用 MV3D_RGBD_SetParam() 接口进行配置。

5.1 常用的相机参数

下表为常用的相机参数，使用时不区分设备类型。

参数名称	参数描述	参数范围	默认值	参数类型	参数说明
CameraWorkingMode	设置相机工作模式	0：传感器标定模式 4：深度图模式 10：体积测量模式	4	Enum-枚举型	通过 MV3D_RGBD_OpenDevice() 接口打开的设备 SDK 会自动将其切到深度图模式
ImageAlign	设置图像坐标系对齐模式	0：关闭坐标系对齐 1：对齐到深度图坐标系 2：对齐到彩色图坐标系	1	Int-整数型	SDK 打开设备时默认开启深度图对齐模式，默认为深度图对齐到 RGB 模式
ImageMode	设置图像模式	2：深度图模式 8：彩色图模式 12：mono 图模式	无	Enum-枚举型	不同设备不同工作模式下支持的图像模式存在差异，以实际情况为准

参数名称	参数描述	参数范围	默认值	参数类型	参数说明
PointCloudOutput	输出点云图像类型	0：关闭点云输出； 1：输出点云； 2：输出带法向量的点云； 3：输出纹理点云； 4：输出带法向量的纹理点云	0	Enum-枚举型	可根据 Mv3dRgbdPointCloudType 进行配置

其他相机参数的具体信息可通过客户端查看。客户端连接相机后，在基本属性处选中具体的某个参数，此时客户端左下方会显示该参数的节点名称、参数类型、数值范围、各数值对应的含义等，如下图所示。

说明

节点名称即 Node Name，是 MV3D_RGBD_GetParam()、MV3D_RGBD_SetParam()以及 MV3D_RGBD_Execute()接口所需要传入的 strKey。

5.2 深度图后处理的算法参数

下表为深度图后处理，杂点滤波相关的算法参数。

参数名称	参数描述	参数范围	默认值	参数类型	参数说明
NoiseFilterEnable	杂点滤除算法使能	TRUE：打开 FALSE：关闭	FALSE	Bool-布尔型	通过该节点开关杂点滤除算法
NoiseFilterWidth	设置杂点滤除宽度	1~51	3	Int-整数型	杂点滤除宽度阈值
NoiseFilterHeight	设置杂点滤除高度	1~51	3	Int-整数型	杂点滤除高度阈值
NoiseFilterType	设置杂点滤除类型	1~2	2	Enum-枚举型	外点滤除方法选择，1-连通域滤除，2-Speckle 散点滤除
NoiseFilterSpeckleDepthDiff	设置散点深度差异	1~65535	500	Int-整数型	散点滤除近邻深度差异阈值，仅在散点滤除类型下生效
NoiseFilterSpeckleArea	设置散点面积	1~10000000	50	Int-整数型	散点滤除像素面积阈值，小于该面积的连通域将被剔除，仅在散点滤除类型下生效

下表为深度图后处理，法向量滤除相关的算法参数。

参数名称	参数描述	参数范围	默认值	参数类型	参数说明
NormalFilterEnable	法向量滤除算法使能	TRUE：打开 FALSE：关闭	FALSE	Bool-布尔型	通过该节点开关法向量滤除算法
NormalFilterWindowWidth	设置法向量滤除窗口大小	3~15	5	Int-整数型	法向量计算窗口大小，窗口越大滤波效果越显著，法向量越平滑
NormalFilterMorphSize	设置形态学窗口大小	0~17	0	Int-整数型	法向量滤除掩膜形态学处理窗口大小，0 表示关闭
NormalFilterZangleType	设置夹角计算方式	0~2	0	Enum-枚举型	0-使用 xy 方向夹角，1-使用 x 方向夹角，2-使用 y 方向夹角
NormalFilterZangleTher	设置滤除角度阈值	0~90	45	Int-整数型	深度图法向量滤除 z 轴夹角阈值，单位为度（角度），将法向量与 z 轴夹角大于阈值的点去除

下表为深度图后处理，深度图延拓相关的算法参数。

参数名称	参数描述	参数范围	默认值	参数类型	参数说明
ExtenEnable	延拓使能	TRUE：打开 FALSE：关闭	FALSE	Bool-布尔型	通过该节点开关深度图延拓算法
ExtenMode	延拓模式	0~3	0	Enum-枚举型	延拓模式：平滑模式（0：X 方向；1：Y 方向；2：XY 方向），细节模式（3：X 方向）
ExtenPointNum	延拓点数	1~100	5	Int-整数型	设置延拓点数

下表为深度图后处理，间隙填充相关的算法参数。

参数名称	参数描述	参数范围	默认值	参数类型	参数说明
GapFillEnable	间隙填充算法使能	TRUE：打开 FALSE：关闭	FALSE	Bool-布尔型	通过该节点开关间隙填充算法
GapFillMode	间隙填充模式	1~3	3	Enum-枚举型	填充模式，1-仅填充 X 方向空点；2-仅填充 y 方向空点，3-同时填充 XY 方向空点
GapFillXNum	设置 X 方向间隙填充数量	0~100	5	Int-整数型	X 方向填充点数量
GapFillYNum	设置 Y 方向间隙填充数量	0~100	5	Int-整数型	Y 方向填充点数量
GapFillHeightTh	设置间隙填充高度阈值	1~10000	1	Int-整数型	空点填充的高度差阈值，空点两侧的点高度差阈值小于一定值才进行填充

下表为深度图后处理，深度图滤波相关的算法参数。

参数名称	参数描述	参数范围	默认值	参数类型	参数说明
RangeFilterEnable	深度图滤波算法使能	TRUE：打开 FALSE：关闭	FALSE	Bool-布尔型	通过该节点开关间隙填充算法
RangeFilterType	设置滤波类型	1~5	1	Enum-枚举型	仅在开启深度图滤波算法后生效
RangeFilterWinWidth	设置滤波窗口宽度	1~501	3	Int-整数型	滤波窗口，窗口越大滤波效果越显著，仅在滤波类型为 1-4 时生效
RangeFilterWinHeight	设置滤波窗口高度	1~501	3	Int-整数型	滤波窗口，窗口越大滤波效果越显著，仅在滤波类型为 1-4 时生效
RangeFilterGuassSigma	设置高斯标准差	1~1000	10	Int-整数型	sigma 越大，高斯模型越接近均值滤波，仅在滤波类型为 1-4 时生效
RangeFilterStddev	设置空域滤波标准差	1~1000	400	Int-整数型	空间滤波标准差，越大越平滑，仅在滤波类型为 5 时生效
RangeFilterDecay	设置空域滤波衰减	1~1000	10	Int-整数型	空间滤波衰减等级，仅在滤波类型为 5 时生效

下表为深度图后处理，区间抽取相关的算法参数。

参数名称	参数描述	参数范围	默认值	参数类型	参数说明
HeightExtractEnable	区间抽取算法使能	TRUE：打开 FALSE：关闭	FALSE	Bool-布尔型	通过该节点开关区间抽取算法
HeightExtractMinDepth	设置最小抽取高度	-65535~2000	0	Int-整数型	抽取的最低高度区间，单位为深度值的单位
HeightExtractMaxDepth	设置最大抽取高度	0~65535	2000	Int-整数型	抽取的最高高度区间，单位为深度值的单位

下表为深度图后处理，深度图变换相关的算法参数。

参数名称	参数描述	参数范围	默认值	参数类型	参数说明
RangeTransformEnable	深度图变换使能	TRUE：打开 FALSE：关闭	FALSE	Bool-布尔型	通过该节点开关深度图变换算法
RangeTransformType	变换类型	1~3	1	Enum-整数型	深度图变换类型，1-上下翻转；2-左右翻转；3-XY 转置

第6章 控制台 Demo

6.1 BasicDemo

该示例程序主要实现相机设备的枚举、打开、取流、数据解析、关闭等功能。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT

g_bExit = False
def work_thread(camera=0,pdata=0,nDataSize=0):
    while True:
        stFrameData=MV3D_RGBD_FRAME_DATA()
        ret=camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
        if ret==0:
            for i in range(0, stFrameData.nImageCount):

print("MV3D_RGBD_FetchFrame[%d]:nFrameNum[%d],nDataLen[%d],nWidth[%d],nHeight[%d]
" % (
              i, stFrameData.stImageData[i].nFrameNum,
stFrameData.stImageData[i].nDataLen, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nHeight))
        else:
            print("no data[0x%x]" % ret)
        if g_bExit == True:
            break

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir,
pointer(stDeviceList.DeviceInfo[0]), 20, nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
```

```
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to
connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnection
Num)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    # ch:获取图像线程 | en:Get image thread
    try:

hthreadhandle=threading.Thread(target=work_thread,args=(camera,None,None))
        hthreadhandle.start()
    except:
        print("error: unable to start thread")

    os.system('pause')
    g_bExit = True

    hthreadhandle.join()

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    sys.exit()
```

6.2 BasicDemo_Callback

该示例程序说明如何通过注册图像数据回调接口获取数据信息。

```
# -- coding: utf-8 --
import msvcrt
import os
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet,
DeviceType_USB, DeviceType_Ethernet_Vir, DeviceType_USB_Vir

FrameInfoCallBack=WINFUNCTYPE(None, c_void_p,c_void_p)

def image_callback(pstFrameData,pUser):
    if pstFrameData==None:
        print("no data")
        return
    stFrameData = cast(pstFrameData, POINTER(MV3D_RGBD_FRAME_DATA)).contents
    if stFrameData.nImageCount != 0:
        for i in range(0, stFrameData.nImageCount):

print("MV3D_RGBD_FetchFrame[%d]:nFrameNum[%d],nDataLen[%d],nWidth[%d],nHeig
ht[%d]" % (
              i, stFrameData.stImageData[i].nFrameNum,
stFrameData.stImageData[i].nDataLen,
              stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nHeight))
    else:
        print("no data")

CALL_BACK_FUN=FrameInfoCallBack(image_callback)

if __name__ == "__main__":

    nDeviceNum = ctypes.c_uint(0)
    nDeviceNum_p = byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret = Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet |
DeviceType_USB | DeviceType_Ethernet_Vir | DeviceType_USB_Vir,
nDeviceNum_p)
    if ret != 0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if nDeviceNum == 0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    # ch:枚举设备信息 | en:Enumerate device information
    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet |
DeviceType_USB | DeviceType_Ethernet_Vir | DeviceType_USB_Vir,
                                pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)
    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)
```

```
        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera = Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to
connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnection
Num)]))
    if ret != 0:
        print("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:注册回调函数 | en:Register the callback function
    camera.MV3D_RGBD_RegisterFrameCallBack(CALL_BACK_FUN,None)

    # ch:开始取流 | en:Start grabbing
    ret = camera.MV3D_RGBD_Start()
    if ret != 0:
        print("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    os.system('pause')

    # ch:停止取流 | en:Stop grabbing
    ret = camera.MV3D_RGBD_Stop()
    if ret != 0:
        print("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret = camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    sys.exit()
```

6.3 DepthValue

该示例程序说明获取到深度图后如何解析深度值。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet,
DeviceType_USB, DeviceType_Ethernet_Vir, DeviceType_USB_Vir,
MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, MV3D_RGBD_OK,ImageType_Depth

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet |
DeviceType_USB | DeviceType_Ethernet_Vir | DeviceType_USB_Vir,
nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet |
DeviceType_USB | DeviceType_Ethernet_Vir | DeviceType_USB_Vir,
pointer(stDeviceList.DeviceInfo[0]), 20, nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to
connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
```

```
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    stFrameData = MV3D_RGBD_FRAME_DATA()
    # ch:获取图像数据 | en:Get image data
    ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
    if MV3D_RGBD_OK == ret:
        print("MV3D_RGBD_FetchFrame success.")
        for i in range(0, stFrameData.nImageCount):
            if ImageType_Depth == stFrameData.stImageData[i].enImageType:
                strMode =
string_at(stFrameData.stImageData[i].pData,stFrameData.stImageData[i].nDataLen)
                # ch:解析深度图中点深度 | en:Parse depth image center depth value
                sValue = struct.unpack('H'*int(len(strMode)/2), strMode)
                nCenterX = int(stFrameData.stImageData[i].nWidth / 2 )
                nCenterY = int(stFrameData.stImageData[i].nHeight / 2)
                print("-------- Depth Image Center: (%d, %d); Depth Value: %d
mm" %(nCenterX, nCenterY, sValue[nCenterX + nCenterY *
stFrameData.stImageData[i].nWidth]))
    else:
        print("MV3D_RGBD_FetchFrame lost frame!")

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    sys.exit()
```

6.4 DumpAllParams

该示例程序说明如何打印设备中的参数信息，例如工作模式等。

```
# -- coding: utf-8 --

import threading
import msvcrt

import ctypes
import time
import os

from ctypes import *

from Mv3dRgbdImport.Mv3dRgbdDefine import *

from Mv3dRgbdImport.Mv3dRgbdApi import *

from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME,
ParamType_Float, ParamType_Int, ParamType_Enum,ParamType_Bool,
CoordinateType_Depth,ParamType_String,
MV3D_RGBD_FLOAT_Z_UNIT,MV3D_RGBD_OK,MV3D_RGBD_INT_WIDTH,
MV3D_RGBD_INT_HEIGHT,MV3D_RGBD_ENUM_WORKINGMODE,MV3D_RGBD_ENUM_PIXELFORMAT,MV3D_RGBD_EN
UM_IMAGEMODE,MV3D_RGBD_FLOAT_GAIN,MV3D_RGBD_FLOAT_FRAMERATE,MV3D_RGBD_ENUM_TRIGGERSELEC
TOR,MV3D_RGBD_ENUM_TRIGGERMODE,MV3D_RGBD_ENUM_TRIGGERSOURCE,
MV3D_RGBD_FLOAT_TRIGGERDELAY, MV3D_RGBD_INT_IMAGEALIGN


def DumpParam(camera = 0, pParamName =""):


    pstValue = MV3D_RGBD_PARAM()
    pstValue_p= byref (pstValue)
    nRet = MV3D_RGBD_OK

    nRet = camera.MV3D_RGBD_GetParam(pParamName, pstValue_p)

    if nRet!= MV3D_RGBD_OK:
        print("MV3D_RGBD_GetParam  pParamName [%s] fail! ret[0x%x]" %pParamName % nRet)
        return

    if ParamType_Int == pstValue.enParamType:

        print("ParamName : %s, Current Value: %d ,Max Value: %d ,Min Value: %d \r\n"
            % (pParamName , pstValue.ParamInfo.stIntParam.nCurValue ,
pstValue.ParamInfo.stIntParam.nMax ,
            pstValue.ParamInfo.stIntParam.nMin))

    elif ParamType_Float == pstValue.enParamType:

        print("ParamName : %s, Current Value: %f ,Max Value: %f ,Min Value: %f \r\n"
        %(pParamName ,pstValue.ParamInfo.stFloatParam.fCurValue ,pstValue.ParamInfo.stFlo
atParam.fMax ,pstValue.ParamInfo.stFloatParam.fMin))

    elif ParamType_Enum == pstValue.enParamType:

            print("ParamName : %s, Current Value: %d ,Supported Number: %d \r\n"
            %(pParamName ,pstValue.ParamInfo.stEnumParam.nCurValue ,pstValue.ParamInfo.st
EnumParam.nSupportedNum))
            print("          %s Enum options :\r\n" %pParamName)
            for  i in range(0,pstValue.ParamInfo.stEnumParam.nSupportedNum):
```

```
          print("          Support Value is [%d]
\r\n" %pstValue.ParamInfo.stEnumParam.nSupportValue[i])

    elif ParamType_Bool == pstValue.enParamType:

        print("ParamName : %s, Current BoolValue: %d
\r\n" %pParamName %pstValue.ParamInfo.bBoolParam)

    elif ParamType_String == pstValue.enParamType:

        print("ParamName : %s, Current String MaxLength: %d,Current String
Value: %s\r\n" %(pParamName ,pstValue.ParamInfo.stStringParam.nMaxLength ,str(pstValue.Pa
ramInfo.stStringParam.chCurValue,encoding = 'ascii')))
    return


def DumpAllParams(pHandle = 0):

    DumpParam(pHandle, "DeviceModelName")

    DumpParam(pHandle, MV3D_RGBD_INT_WIDTH)

    DumpParam(pHandle, MV3D_RGBD_INT_HEIGHT)

    DumpParam(pHandle, MV3D_RGBD_ENUM_WORKINGMODE)

    DumpParam(pHandle, MV3D_RGBD_ENUM_PIXELFORMAT)

    DumpParam(pHandle, MV3D_RGBD_ENUM_IMAGEMODE)

    DumpParam(pHandle, MV3D_RGBD_FLOAT_GAIN)

    DumpParam(pHandle, MV3D_RGBD_FLOAT_EXPOSURETIME)

    DumpParam(pHandle, MV3D_RGBD_FLOAT_FRAMERATE)

    DumpParam(pHandle, MV3D_RGBD_ENUM_TRIGGERSELECTOR)

    DumpParam(pHandle, MV3D_RGBD_ENUM_TRIGGERMODE)

    DumpParam(pHandle, MV3D_RGBD_ENUM_TRIGGERSOURCE)

    DumpParam(pHandle, MV3D_RGBD_FLOAT_TRIGGERDELAY)

    DumpParam(pHandle, MV3D_RGBD_INT_IMAGEALIGN)

    return MV3D_RGBD_OK


if __name__ == "__main__":

    nDeviceNum=ctypes.c_uint(0)

    nDeviceNum_p=byref(nDeviceNum)

    # ch:获取设备数量 | en:Get device number

    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)

    if  ret!=0:

        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
```

```
        os.system('pause')

        sys.exit()

    if  nDeviceNum==0:

        print("find no device!")

        os.system('pause')

        sys.exit()

    print("Find devices numbers:", nDeviceNum.value)


    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()

    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)


    for i in range(0, nDeviceNum.value):

        print("\ndevice: [%d]" % i)

        strModeName = ""

        for per in stDeviceList.DeviceInfo[i].chModelName:

            strModeName = strModeName + chr(per)

        print("device model name: %s" % strModeName)


        strSerialNumber = ""

        for per in stDeviceList.DeviceInfo[i].chSerialNumber:

            strSerialNumber = strSerialNumber + chr(per)

        print("device SerialNumber: %s" % strSerialNumber)


    # ch:创建相机示例 | en:Create a camera instance

    camera=Mv3dRgbd()

    nConnectionNum = input("please input the number of the device to connect:")

    if int(nConnectionNum) >= nDeviceNum.value:

        print("intput error!")

        os.system('pause')

        sys.exit()


    # ch:打开设备 | en:Open device

    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))

    if ret != 0:
```

```
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    ret = DumpAllParams(camera)

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    sys.exit()
```

6.5 FetchFrame

该示例程序说明如何通过轮询的方式获取数据信息。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import tkinter as tk
from tkinter import ttk
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT

g_bExit = False
def work_thread(camera=0,lable=0,nDataSize=0):
    while True:
        stFrameData=MV3D_RGBD_FRAME_DATA()
        ret=camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
        if ret==0:
            for i in range(0, stFrameData.nImageCount):

print("MV3D_RGBD_FetchFrame[%d]:nFrameNum[%d],nDataLen[%d],nWidth[%d],nHeight[%d]" % (
                i, stFrameData.stImageData[i].nFrameNum,
stFrameData.stImageData[i].nDataLen, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nHeight))
            if g_bExit != True:
                ret =
camera.MV3D_RGBD_DisplayImage(pointer(stFrameData.stImageData[0]),lable.winfo_id())
                if ret!=0:
                    print("DisplayImage[0x%x]" % ret)
        else:
            print("no data[0x%x]" % ret)
        if g_bExit == True:
            break

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
```

```
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    window = tk.Tk()
    window.title('Login & Register Window')
    window.geometry('768x512')
    lable = tk.Label(window,compound='center',width='768',height='512')
    # ch:获取图像线程 | en:Get image thread
    try:
        hthreadhandle=threading.Thread(target=work_thread,args=(camera,lable ,None))
        hthreadhandle.start()
    except:
        print("error: unable to start thread")
    lable.pack()
    window.mainloop()
    g_bExit = True

    hthreadhandle.join()

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
```

6.6 ForceDeviceIP

该示例程序说明如何设置当前设备的 IP。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, Mv3dRgbdIpCfgMode

def GetDestIP():
    stIpCfgInfo = MV3D_RGBD_IP_CONFIG()
    print("Please enter the network mode of the camera to be configure \n")
    nIPCfgMode  = int(input("[1] STATIC [2] DHCP [3] LLA :"))

    if (1 == nIPCfgMode):
        pNewIP = input("Please enter the camera IP to be set: ").encode("ascii")
        for i in range(0, len(pNewIP)):
            stIpCfgInfo.chDestIp[i] = pNewIP[i]

        pNewNetMask = input("Please enter the camera Netmask to be set: ").encode("ascii")
        for i in range(0, len(pNewNetMask)):
            stIpCfgInfo.chDestNetMask[i] = pNewNetMask[i]

        pNewDestGateWay = input("Please enter the camera Gateway to be set:
").encode("ascii")
        for i in range(0, len(pNewDestGateWay)):
            stIpCfgInfo.chDestGateWay[i] = pNewDestGateWay[i]

    elif (3 == nIPCfgMode):
        nIPCfgMode = IpCfgMode_LLA

    elif ((0 >=  nIPCfgMode)  or (3 < nIPCfgMode)):
        print("Enter error ! Get it error!")
        return 0, stIpCfgInfo

    stIpCfgInfo.enIPCfgMode = Mv3dRgbdIpCfgMode(nIPCfgMode)
    return 1, stIpCfgInfo

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
```

```
net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    stIpCfgInfo = MV3D_RGBD_IP_CONFIG()

    ret, stIpCfgInfo = GetDestIP()
    if (0 == ret ):
        print("GetDestIP fail!")
        sys.exit()
    else:
        ret  =
camera.MV3D_RGBD_SetIpConfig(stDeviceList.DeviceInfo[1].chSerialNumber,stIpCfgInfo)
        if ret  == 0:
            print("MV3D_RGBD_SetIpConfig success!")
        else:
            print("MV3D_RGBD_SetIpConfig fail! ret[0x%x]" % ret)
            sys.exit()

    print("Main done !")
    os.system('pause')
    sys.exit()
```

6.7 GetPointCloudImage

该示例程序说明如何设置点云输出节点，获取点云后如何解析点云数据并保存点云图像。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, MV3D_RGBD_OK,ImageType_Depth, \

PointCloudType_Common,PointCloudType_Normals ,PointCloudType_Texture,PointCloudType_Textu
re_Normals,MV3D_RGBD_ENUM_POINT_CLOUD_OUTPUT, \
        ImageType_PointCloud, ImageType_PointCloudWithNormals,
ImageType_TexturedPointCloud, ImageType_TexturedPointCloudWithNormals,
PointCloudFileType_PLY

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
```

```
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:设置点云输出节点 | en:Set point cloud output node
    stParam = MV3D_RGBD_PARAM()
    stParam.enParamType = ParamType_Enum
    stParam.ParamInfo.stEnumParam.nCurValue = PointCloudType_Common
    ret = camera.MV3D_RGBD_SetParam(MV3D_RGBD_ENUM_POINT_CLOUD_OUTPUT, pointer(stParam))

    if MV3D_RGBD_OK != ret:
        print ("SetParam fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    print ("Set point cloud output success.")

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    stFrameData = MV3D_RGBD_FRAME_DATA()
    # ch:获取图像数据 | en:Get image data
    ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
    if MV3D_RGBD_OK == ret:
        print("MV3D_RGBD_FetchFrame success.")
        for i in range(0, stFrameData.nImageCount):
            # ch:解析点云数据 | en:Parse point cloud data
            if ImageType_PointCloud == stFrameData.stImageData[i].enImageType:
                print("Get point cloud succeed: framenum (%d) height(%d) width(%d)
len(%d)!" %(stFrameData.stImageData[i].nFrameNum,
                        stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, stFrameData.stImageData[i].nDataLen))
                # ch:保存点云图像 | en:Save point cloud image
                chFileName =
"[{:d}]_PointCloudImage".format(stFrameData.stImageData[i].nFrameNum)
                ret =
camera.MV3D_RGBD_SavePointCloudImage(pointer(stFrameData.stImageData[i]),
PointCloudFileType_PLY, chFileName)
                if MV3D_RGBD_OK == ret:
                    print("Save %s success!" % chFileName)
    else:
        print("MV3D_RGBD_FetchFrame lost frame!")

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
```

```
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    sys.exit()
```

6.8 ImageAlign

该示例程序说明如何设置图像对齐模式，获取对齐到同一坐标系的深度图和彩色图。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, MV3D_RGBD_OK,ImageType_Depth, \
        MV3D_RGBD_CAMERA_PARAM,MV3D_RGBD_INT_IMAGEALIGN

def PrintCameraParam(stCameraParam):
    print("-------- Depth Calib Info --------")
    print("Depth Width: %d" % stCameraParam.stDepthCalibInfo.nWidth)
    print("Depth Height: %d" % stCameraParam.stDepthCalibInfo.nHeight)
    print("--------")
    for i in range(0,9):
        print("Depth Intrinsic[%d]: %.10f" % (i,
stCameraParam.stDepthCalibInfo.stIntrinsic.fData[i]))
    print("--------")
    for i in range(0,12):
        print("Depth Distortion[%d]: %.10f" % ( i,
stCameraParam.stDepthCalibInfo.stDistortion.fData[i]))

    print("-------- Rgb Calib Info --------")
    print("Rgb Width: %d" % stCameraParam.stRgbCalibInfo.nWidth)
    print("Rgb Height: %d" % stCameraParam.stRgbCalibInfo.nHeight)
    print("--------")
    for i in range(0,9):
        print("Rgb Intrinsic[%d]: %.10f" % (i,
stCameraParam.stRgbCalibInfo.stIntrinsic.fData[i]))
    print("--------")
    for i in range(0,12):
        print("Rgb Distortion[%d]: %.10f" % (i,
stCameraParam.stRgbCalibInfo.stDistortion.fData[i]))
    print("-------- Depth to Rgb Extrinsic --------")
    for i in range(0,16):
        print("Depth to Rgb Extrinsic[%d]: %.10f" % (i,
stCameraParam.stDepth2RgbExtrinsic.fData[i]))
    print("--------")
    return

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
```

```
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:设置图像对齐深度图坐标系模式 | en:Set image align depth coordinate mode
    stParam = MV3D_RGBD_PARAM()
    stParam.enParamType = ParamType_Int
    stParam.ParamInfo.stIntParam.nCurValue = 1
    ret = camera.MV3D_RGBD_SetParam(MV3D_RGBD_INT_IMAGEALIGN, pointer(stParam))

    if MV3D_RGBD_OK != ret:
        print ("SetParam fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    print("Set image align success.")

    # ch:获取传感器标定参数 | en:Get sensor calib param
    stCameraParam = MV3D_RGBD_CAMERA_PARAM()
    ret = camera.MV3D_RGBD_GetCameraParam(pointer(stCameraParam))
    if MV3D_RGBD_OK != ret:
        print ("MV3D_RGBD_GetCameraParam fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    print("Get camera param success.")

    PrintCameraParam(stCameraParam)

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
```

```
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    stFrameData = MV3D_RGBD_FRAME_DATA()
    # ch:获取图像数据 | en:Get image data
    ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
    if MV3D_RGBD_OK == ret:
        print("MV3D_RGBD_FetchFrame success.")
    else:
        print("MV3D_RGBD_FetchFrame lost frame!")

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    sys.exit()
```

6.9 ImageCoordinateTrans

该示例程序说明如何将获取到的深度图转换到彩色图坐标系。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, MV3D_RGBD_OK,ImageType_Depth, \
        MV3D_RGBD_CAMERA_PARAM,MV3D_RGBD_INT_IMAGEALIGN

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
```

```
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:关闭图像对齐深度图坐标系模式 | en:Close image align depth coordinate mode
    stParam = MV3D_RGBD_PARAM()
    stParam.enParamType = ParamType_Int
    stParam.ParamInfo.stIntParam.nCurValue = 0
    ret = camera.MV3D_RGBD_SetParam(MV3D_RGBD_INT_IMAGEALIGN, pointer(stParam))

    if MV3D_RGBD_OK != ret:
        print ("SetParam fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    print("Close image align success.")

    ret = camera.MV3D_RGBD_GetParam(MV3D_RGBD_FLOAT_Z_UNIT, pointer(stParam))
    if MV3D_RGBD_OK != ret:
        print ("GetParam fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    print("Get z-unit success.")
    fZunit = stParam.ParamInfo.stFloatParam.fCurValue

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    stFrameData = MV3D_RGBD_FRAME_DATA()
    # ch:获取图像数据 | en:Get image data
    ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
    if MV3D_RGBD_OK == ret:
        if stFrameData.nValidInfo == False:
            print("MV3D_RGBD_FetchFrame success.")
            for i in range(0, stFrameData.nImageCount):
                if ImageType_Depth == stFrameData.stImageData[i].enImageType:
                    print("depth raw image: framenum (%d) height(%d) width(%d) len(%d)
coordinate(%d)!" %(stFrameData.stImageData[i].nFrameNum,
                           stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, stFrameData.stImageData[i].nDataLen,
stFrameData.stImageData[i].enCoordinateType))
                    # ch:获取传感器标定信息 | en:Get sensor calib info
                    stCameraParam = MV3D_RGBD_CAMERA_PARAM()
                    ret = camera.MV3D_RGBD_GetCameraParam(pointer(stCameraParam))
                    if MV3D_RGBD_OK != ret:
                        print ("MV3D_RGBD_GetCameraParam fail! ret[0x%x]" % ret)
                        camera.MV3D_RGBD_CloseDevice()
                        os.system('pause')
                        sys.exit()
                    print("Get camera param success.")

                    # ch:将深度图转换到RGB坐标系下 | en:Convert depth map to rgb coordinate
                    stDepthConvImg = MV3D_RGBD_IMAGE_DATA()
                    ret =
camera.MV3D_RGBD_ImageCoordinateTrans(pointer(stFrameData.stImageData[i]), fZunit,
pointer(stDepthConvImg),  pointer(stCameraParam))
```

```
                if MV3D_RGBD_OK == ret:
                    print("depth convert image: framenum (%d) height(%d) width(%d)
len(%d) coordinate(%d)!" % (stDepthConvImg.nFrameNum,
                        stDepthConvImg.nHeight, stDepthConvImg.nWidth,
stDepthConvImg.nDataLen, stFrameData.stImageData[i].enCoordinateType))
                else:
                    print("MV3D_RGBD_ImageCoordinateTrans failed...sts[%#x]", ret)
        else:
            print("stFrameData.nValidInfo is true!")
    else:
        print("MV3D_RGBD_FetchFrame lost frame!")

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    sys.exit()
```

6.10 ListDevices

该示例程序说明如何打印并保存当前枚举出的设备信息。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT,IpCfgMode_Static,IpCfgMode_DHCP,IpCfgMode_LLA

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        if (DeviceType_Ethernet == stDeviceList.DeviceInfo[i].enDeviceType) or
(DeviceType_Ethernet_Vir == stDeviceList.DeviceInfo[i].enDeviceType):
            print("SerialNum[%s] IP[%s] name[%s] DeviceVersion[%s].\r\nManufacturerName[%s]
UserDefinedName[%s] MacAddress[%x:%x:%x:%x:%x:%x] CurrentSubNetMask[%s] DefultGateWay[%s]
NetExport[%s] Type[%d].\r\n"
              %(bytes(bytearray(stDeviceList.DeviceInfo[i].chSerialNumber)).decode('ascii'
),
bytes(bytearray(stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chCurrentIp)).decode('as
cii'),bytes(bytearray(stDeviceList.DeviceInfo[i].chModelName)).decode('ascii') ,bytes(byt
earray(stDeviceList.DeviceInfo[i].chDeviceVersion)).decode('ascii') ,

bytes(bytearray(stDeviceList.DeviceInfo[i].chManufacturerName)).decode('ascii'),
bytes(bytearray(stDeviceList.DeviceInfo[i].chUserDefinedName)).decode('ascii'),
              stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chMacAddress[0],
stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chMacAddress[1],
stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chMacAddress[2],
              stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chMacAddress[3],
stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chMacAddress[4],
stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chMacAddress[5],

bytes(bytearray(stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chCurrentSubNetMask)).de
code('ascii'),bytes(bytearray(stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chDefultGa
teWay)).decode('ascii') ,bytes(bytearray(stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo
.chNetExport)).decode('ascii') ,
```

```
              (stDeviceList.DeviceInfo[i].nDevTypeInfo & 0xff000000) >> 24))

          if IpCfgMode_Static &
stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.enIPCfgMode:
              print("IPCfgMode[STATIC]")
          if IpCfgMode_DHCP &
stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.enIPCfgMode:
              print("IPCfgMode[DHCP]")
          if IpCfgMode_LLA &
stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.enIPCfgMode:
              print("IPCfgMode[LLA]")

       elif (DeviceType_USB == stDeviceList.DeviceInfo[i].enDeviceType or
DeviceType_USB_Vir == stDeviceList.DeviceInfo[i].enDeviceType):

          print("SerialNum[%s] name[%s] DeviceVersion[%s].\r\nManufacturerName[%s]
UserDefinedName[%s] VendorId[%d] ProductId[%d] enUsbProtocol[%d] DeviceGUID[%s]
Type[%d].\r\n" %

(bytes(bytearray(stDeviceList.DeviceInfo[i].chSerialNumber)).decode('ascii'),
bytes(bytearray(stDeviceList.DeviceInfo[i].chModelName)).decode('ascii'),bytes(bytearray(
stDeviceList.DeviceInfo[i].chDeviceVersion)).decode('ascii') ,

bytes(bytearray(stDeviceList.DeviceInfo[i].chManufacturerName)).decode('ascii'),
bytes(bytearray(stDeviceList.DeviceInfo[i].chUserDefinedName)).decode('ascii'),
              stDeviceList.DeviceInfo[i].SpecialInfo.stUsbInfo.nVendorId,
stDeviceList.DeviceInfo[i].SpecialInfo.stUsbInfo.nProductId,
              stDeviceList.DeviceInfo[i].SpecialInfo.stUsbInfo.enUsbProtocol,
bytes(bytearray(stDeviceList.DeviceInfo[i].SpecialInfo.stUsbInfo.chDeviceGUID)).decode('a
scii'),
              (stDeviceList.DeviceInfo[i].nDevTypeInfo & 0xff000000) >> 24))

       print("************************************************\r\n",)

    i = 0
    with open("CurrentDeviceInfo.txt","w") as f:
       if (DeviceType_Ethernet == stDeviceList.DeviceInfo[i].enDeviceType) or
(DeviceType_Ethernet_Vir == stDeviceList.DeviceInfo[i].enDeviceType):
          chDevInfo = "SerialNum[{:s}] IP[{:s}]  name[{:s}]
DeviceVersion[{:s}].ManufacturerName[{:s}] UserDefinedName[{:s}]
MacAddress[{:02x}:{:02x}:{:02x}:{:02x}:{:02x}:{:02x}] CurrentSubNetMask[{:s}]
DefultGateWay[{:s}] NetExport[{:s}] Type[{:d}].".format(

bytes(bytearray(stDeviceList.DeviceInfo[i].chSerialNumber)).decode('ascii').strip(b'\x00'
.decode()),
bytes(bytearray(stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chCurrentIp)).decode('as
cii').strip(b'\x00'.decode()),bytes(bytearray(stDeviceList.DeviceInfo[i].chModelName)).de
code('ascii').strip(b'\x00'.decode()) ,bytes(bytearray(stDeviceList.DeviceInfo[i].chDevic
eVersion)).decode('ascii').strip(b'\x00'.decode()) ,

bytes(bytearray(stDeviceList.DeviceInfo[i].chManufacturerName)).decode('ascii').strip(b'\
x00'.decode()),
bytes(bytearray(stDeviceList.DeviceInfo[i].chUserDefinedName)).decode('ascii').strip(b'\x
00'.decode()),stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chMacAddress[0],
stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chMacAddress[1],
stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chMacAddress[2],
              stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chMacAddress[3],
stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chMacAddress[4],
stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chMacAddress[5],

bytes(bytearray(stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chCurrentSubNetMask)).de
code('ascii').strip(b'\x00'.decode()),bytes(bytearray(stDeviceList.DeviceInfo[i].SpecialI
nfo.stNetInfo.chDefultGateWay)).decode('ascii').strip(b'\x00'.decode()) ,bytes(bytearray(
stDeviceList.DeviceInfo[i].SpecialInfo.stNetInfo.chNetExport)).decode('ascii').strip(b'\x
00'.decode()) ,
              (stDeviceList.DeviceInfo[i].nDevTypeInfo & 0xff000000) >> 24)
```

```
        elif (DeviceType_USB == stDeviceList.DeviceInfo[i].enDeviceType or
DeviceType_USB_Vir == stDeviceList.DeviceInfo[i].enDeviceType):
            chDevInfo = "SerialNum[{:s}] name[{:s}]
DeviceVersion[{:s}].ManufacturerName[{:s}] UserDefinedName[{:s}] VendorId[{:d}]
ProductId[{:d}] enUsbProtocol[{:d}] DeviceGUID[{:s}] Type[{:d}].".format(

bytes(bytearray(stDeviceList.DeviceInfo[i].chSerialNumber)).decode('ascii').strip(b'\x00'
.decode()),
bytes(bytearray(stDeviceList.DeviceInfo[i].chModelName)).decode('ascii').strip(b'\x00'.de
code()),bytes(bytearray(stDeviceList.DeviceInfo[i].chDeviceVersion)).decode('ascii').stri
p(b'\x00'.decode()) ,

bytes(bytearray(stDeviceList.DeviceInfo[i].chManufacturerName)).decode('ascii').strip(b'\
x00'.decode()),
bytes(bytearray(stDeviceList.DeviceInfo[i].chUserDefinedName)).decode('ascii').strip(b'\x
00'.decode()),stDeviceList.DeviceInfo[i].SpecialInfo.stUsbInfo.nVendorId,
stDeviceList.DeviceInfo[i].SpecialInfo.stUsbInfo.nProductId,
              stDeviceList.DeviceInfo[i].SpecialInfo.stUsbInfo.enUsbProtocol,
bytes(bytearray(stDeviceList.DeviceInfo[i].SpecialInfo.stUsbInfo.chDeviceGUID)).decode('a
scii').strip(b'\x00'.decode()),
              (stDeviceList.DeviceInfo[i].nDevTypeInfo & 0xff000000) >> 24)
        f.write(chDevInfo)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()

    # ch:打开设备 | en:Open device
    ret = camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(i)]))

    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    print("Main done !")
    os.system('pause')
    sys.exit()
```

6.11 LoopDetect

该示例程序说明发生异常情况后，如何保存图像。

```
# -- coding: utf-8 --
import msvcrt
import os
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir,
MV3D_RGBD_OK,DevException_Disconnect,ImageType_Depth,ImageType_RGB8_Planar

device_offline = False
ExceptionInfoCallBack=WINFUNCTYPE(None, c_void_p,c_void_p)

def exception_callback(pstExceptionData, userdata):
    if pstExceptionData!=None:
        stExceptionData = cast(pstExceptionData,
POINTER(MV3D_RGBD_EXCEPTION_INFO)).contents
        if DevException_Disconnect == stExceptionData.enExceptionId:
            device_offline = True
            print("=== Event Callback: Device Offline!")


CALL_BACK_FUN=ExceptionInfoCallBack(exception_callback)

if __name__ == "__main__":

    nDeviceNum = ctypes.c_uint(0)
    nDeviceNum_p = byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret = Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if ret != 0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if nDeviceNum == 0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)
    bLoop_exit = False
    while(bLoop_exit == False):

        # ch:枚举设备信息 | en:Enumerate device information
        stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
        net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir,
                                    pointer(stDeviceList.DeviceInfo[0]), 20, nDeviceNum_p)
        for i in range(0, nDeviceNum.value):
            print("\ndevice: [%d]" % i)
            strModeName = ""
            for per in stDeviceList.DeviceInfo[i].chModelName:
                strModeName = strModeName + chr(per)
            print("device model name: %s" % strModeName)

            strSerialNumber = ""
            for per in stDeviceList.DeviceInfo[i].chSerialNumber:
                strSerialNumber = strSerialNumber + chr(per)
            print("device SerialNumber: %s" % strSerialNumber)

        # ch:创建相机示例 | en:Create a camera instance
```

```
        camera = Mv3dRgbd()
        nConnectionNum = input("please input the number of the device to connect:")
        if int(nConnectionNum) >= nDeviceNum.value:
            print("intput error!")
            os.system('pause')
            sys.exit()

        # ch:打开设备 | en:Open device
        ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
        if ret != 0:
            print("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
            os.system('pause')
            sys.exit()

        # ch:注册回调函数 | en:Register the callback function
        camera.MV3D_RGBD_RegisterExceptionCallBack(CALL_BACK_FUN, None)

        # ch:开始取流 | en:Start grabbing
        ret = camera.MV3D_RGBD_Start()
        if ret != 0:
            print("start fail! ret[0x%x]" % ret)
            camera.MV3D_RGBD_CloseDevice()
            os.system('pause')
            sys.exit()

        bExit_main = False
        bSaveFrame = False
        stFrameData = MV3D_RGBD_FRAME_DATA()
        while bExit_main == False:
            # ch:获取图像数据 | en:Get image data
            ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData),5000)
            if MV3D_RGBD_OK == ret:
                for i in range(0, stFrameData.nImageCount):
                    print("MV3D_RGBD_FetchFrame success: framenum (%d) height(%d) width(%d)
len (%d)!\r\n" %(stFrameData.stImageData[i].nFrameNum,
                    stFrameData.stImageData[i].nHeight, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nDataLen))

                    if ((True == bSaveFrame) and (stFrameData.stImageData[i].pData is None)
== False):
                        # ch:保存RAW图像 | en:Save raw image
                        if ImageType_Depth == stFrameData.stImageData[i].enImageType:
                            filename =
"Depth_Frame[{:d}]_Width[{:d}]_Height[{:d}].raw".format(stFrameData.stImageData[i].nFrame
Num, stFrameData.stImageData[i].nWidth, stFrameData.stImageData[i].nHeight)
                            strModeName =
string_at(stFrameData.stImageData[i].pData,stFrameData.stImageData[i].nDataLen)
                        elif ImageType_RGB8_Planar == stFrameData.stImageData[i].enImageType:
                            filename =
"RGB_Frame[{:d}]_Width[{:d}]_Height[{:d}].raw".format(stFrameData.stImageData[i].nFrameNu
m, stFrameData.stImageData[i].nWidth, stFrameData.stImageData[i].nHeight)
                            strModeName =
string_at(stFrameData.stImageData[i].pData,stFrameData.stImageData[i].nDataLen)
                        with open(filename,"wb") as f:
                            f.write(strModeName)

            if True == device_offline:
                print("Found device offline")
                break

            print("Please enter the action to be performed:\n")
            key = input("[q]Exit Current Frame [s]Save Photo [x]Exit Dev \n")
            if key == 'q':
                bExit_main = True
```

```
        if key == 's':
            bSaveFrame = True

        if key == 'x':
            bExit_main = True
            bLoop_exit = True

    if device_offline:
        print("device offline release resource")
    else:
        print("normal exit")



    # ch:停止取流 | en:Stop grabbing
    ret = camera.MV3D_RGBD_Stop()
    if ret != 0:
        print("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret = camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

print("Main done !")
sys.exit()
```

6.12 MapDepthToColor

该示例程序说明如何设置深度图转伪彩图像。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, MV3D_RGBD_OK,ImageType_Depth, FileType_BMP, \
       MV3D_RGBD_CAMERA_PARAM,MV3D_RGBD_INT_IMAGEALIGN,ConvertColorMapMode_Rainbow

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
       print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
       os.system('pause')
       sys.exit()
    if  nDeviceNum==0:
       print("find no device!")
       os.system('pause')
       sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
       print("\ndevice: [%d]" % i)
       strModeName = ""
       for per in stDeviceList.DeviceInfo[i].chModelName:
          strModeName = strModeName + chr(per)
       print("device model name: %s" % strModeName)

       strSerialNumber = ""
       for per in stDeviceList.DeviceInfo[i].chSerialNumber:
          strSerialNumber = strSerialNumber + chr(per)
       print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
       print("intput error!")
       os.system('pause')
       sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
```

```
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:配置伪彩图参数 | en:Config parameters of depth convert to color
    stConvertParam = MV3D_RGBD_CONVERT_COLOR_PAPRAM()
    stConvertParam.enConvertColorMapMode = ConvertColorMapMode_Rainbow


    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    stFrameData = MV3D_RGBD_FRAME_DATA()
    # ch:获取图像数据 | en:Get image data
    ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
    if MV3D_RGBD_OK == ret:
        if stFrameData.nValidInfo == False:
            print("MV3D_RGBD_FetchFrame success.")
            for i in range(0, stFrameData.nImageCount):
                if ImageType_Depth == stFrameData.stImageData[i].enImageType:
                    print("depth raw image: framenum (%d) height(%d) width(%d) len(%d)
coordinate(%d)!" %(stFrameData.stImageData[i].nFrameNum,
                            stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, stFrameData.stImageData[i].nDataLen,
stFrameData.stImageData[i].enCoordinateType))
                    stConvertColorImage = MV3D_RGBD_IMAGE_DATA()
                    # ch:将深度图转化为伪彩图 | en:Convert depth to color image
                    ret =
camera.MV3D_RGBD_MapDepthToColor(pointer(stFrameData.stImageData[i]),
pointer(stConvertParam), pointer(stConvertColorImage))
                    if MV3D_RGBD_OK == ret:
                        print("Map depth image %d to color image success! Image height[%d]
width[%d] " %(stConvertColorImage.nFrameNum, stConvertColorImage.nHeight,
stConvertColorImage.nWidth))
                        # ch:保存转化后的图片 | en:Save convert color image
                        ret = camera.MV3D_RGBD_SaveImage(pointer(stConvertColorImage),
FileType_BMP, "ConvertImage")
                        if MV3D_RGBD_OK == ret:
                            print("Save convert image success.")
                        else:
                            print("Save convert image failed...sts[%#x]" % ret)
                    else:
                        print("MV3D_RGBD_MapDepthToColor failed...sts[%#x]" % ret)
        else:
            print("stFrameData.nValidInfo is true!")
    else:
        print("MV3D_RGBD_FetchFrame lost frame!")

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
```

```
        sys.exit()

    sys.exit()
```

6.13 MapPointCloudToUV

该示例程序说明如何通过点云数据获取到 UV 坐标，并渲染出纹理点云。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
CoordinateType_RGB, MV3D_RGBD_FLOAT_Z_UNIT,ImageType_Depth, ImageType_RGB8_Planar,
ImageType_PointCloud, \
    PointCloudType_Common,PointCloudType_Normals ,PointCloudType_Texture,
PointCloudType_Texture_Normals, MV3D_RGBD_ENUM_POINT_CLOUD_OUTPUT

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
```

```
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:设置点云输出节点 | en:Set point cloud output node
    stParam = MV3D_RGBD_PARAM()
    stParam.enParamType = ParamType_Enum
    stParam.ParamInfo.stEnumParam.nCurValue = PointCloudType_Common
    ret = camera.MV3D_RGBD_SetParam(MV3D_RGBD_ENUM_POINT_CLOUD_OUTPUT, pointer(stParam))

    if 0!=ret:
        print ("SetParam fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    print ("Set point cloud output success.")

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    stFrameData = MV3D_RGBD_FRAME_DATA()
    stCalibInfo = MV3D_RGBD_CALIB_INFO()
    bCalibInfo = 0
    # ch:获取图像数据 | en:Get image data
    ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
    if 0 == ret:
        print("MV3D_RGBD_FetchFrame success.")
        for i in range(0, stFrameData.nImageCount):
            if ImageType_PointCloud == stFrameData.stImageData[i].enImageType:
                pstPointCloudImage = stFrameData.stImageData[i]
                print("Get point cloud succeed: framenum (%d) height(%d) width(%d)  len
(%d)!" %( pstPointCloudImage.nFrameNum,
                        pstPointCloudImage.nHeight, pstPointCloudImage.nWidth,
pstPointCloudImage.nDataLen))
                # ch:获取uv坐标 | en: Get uv coordinate
                stUvMap = MV3D_RGBD_UV_DATA()
                if 0==bCalibInfo:
                    ret =
camera.MV3D_RGBD_GetCalibInfo(stFrameData.stImageData[i].enCoordinateType,
pointer(stCalibInfo))
                    if 0!=ret:
                        print("Get calib info failed...")
                        break
                    else:
                        bCalibInfo = 1
                ret =
camera.MV3D_RGBD_MapPointCloudToUV(pointer(stFrameData.stImageData[i]),
pointer(stCalibInfo), pointer(stUvMap))
                if 0==ret:
                    strMode = string_at(stUvMap.pData, stUvMap.nDataLen)
                    sValue = struct.unpack('f'*int(pstPointCloudImage.nHeight *
pstPointCloudImage.nWidth * 2), strMode)
                    with open("UV_data.txt","w") as f:
                        for j in range(0, pstPointCloudImage.nHeight *
pstPointCloudImage.nWidth):
                            strUVValue = "U:{:f},V:{:f} \n".format(sValue[j * 2],sValue[j * 2
+ 1])
```

```
                        f.write(strUVValue)
                else:
                    print("Get uv map failed...")
            # ch:解析彩色图纹理数据 | en: Parse color texture data
            if ImageType_RGB8_Planar == stFrameData.stImageData[i].enImageType:
                print("Get texture color image succeed: framenum (%d) height(%d) width(%d)
len(%d)!" %(stFrameData.stImageData[i].nFrameNum,
                    stFrameData.stImageData[i].nHeight, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nDataLen))
    else:
        print("MV3D_RGBD_FetchFrame lost frame!")

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    print ("Main done")
    sys.exit()
```

6.14 MultiDevice

该示例程序说明多个设备如何通过轮询的方式获取数据信息。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, MV3D_RGBD_OK,ImageType_Depth,MV3D_RGBD_ENUM_POINT_CLOUD_OUTPUT

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    nLinkCount = int(input("Please enter the number of cameras to be connected:"))

    while nLinkCount > nDeviceNum.value or 0 >= nLinkCount:
        nLinkCount = input("The input is greater than the total number of devices.Please
input again! :")

    nIndexArray = []

    for i in range(0,nLinkCount) :
        nIndex = int(input("Please enter the Index number of camera to be connected:"))
        if ((nDeviceNum.value  <= nIndex) or (0 > nIndex)):
            print("enter error!\r\n")
        else:
```

```
            nIndexArray.append(nIndex)
            continue

    cameraArray=[]
    for i in range(0, nLinkCount) :
        # ch:创建相机示例 | en:Create a camera instance
        camera=Mv3dRgbd()
        # ch:打开设备 | en:Open device
        ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nIndexArray[i])]))
        if ret != 0:
            print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
            os.system('pause')
            sys.exit()

         # ch:开始取流 | en:Start grabbing
        ret=camera.MV3D_RGBD_Start()
        if ret != 0:
            print ("start fail! ret[0x%x]" % ret)
            camera.MV3D_RGBD_CloseDevice()
            os.system('pause')
            sys.exit()

        cameraArray.append(camera)


    stFrameData = MV3D_RGBD_FRAME_DATA()
    # ch:获取图像数据 | en:Get image data

    for i in range(0, nLinkCount):
        camera = cameraArray[i]
        ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
        if MV3D_RGBD_OK == ret:
            print("Camera[%d] MV3D_RGBD_FetchFrame success." % nIndexArray[i])
            for j in range(0, stFrameData.nImageCount):
                print("Cam:%d MV3D_RGBD_FetchFrame success: framenum (%d) height(%d)
width(%d)  len (%d)!"% (nIndexArray[i], stFrameData.stImageData[j].nFrameNum,
                stFrameData.stImageData[j].nHeight, stFrameData.stImageData[j].nWidth,
stFrameData.stImageData[j].nDataLen))

    print("All Cam stop work.")
    for i in range(0, nLinkCount):
        print("Camera[%d] stop work." % nIndexArray[i])
        camera = cameraArray[i]
        # ch:停止取流 | en:Stop grabbing
        ret=camera.MV3D_RGBD_Stop()
        if ret != 0:
            print ("stop fail! ret[0x%x]" % ret)
            os.system('pause')
            sys.exit()

        # ch:销毁句柄 | en:Destroy the device handle
        ret=camera.MV3D_RGBD_CloseDevice()
        if ret != 0:
            print ("CloseDevice fail! ret[0x%x]" % ret)
            os.system('pause')
            sys.exit()

    print ("Main done")
    sys.exit()
```

6.15 Point3D

该示例程序说明如何将获取到的深度图转换为点云图。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, MV3D_RGBD_OK,ImageType_Depth

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
```

```
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    stFrameData = MV3D_RGBD_FRAME_DATA()
    # ch:获取图像数据 | en:Get image data
    ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
    if MV3D_RGBD_OK == ret:
        print("MV3D_RGBD_FetchFrame success.")
        for i in range(0, stFrameData.nImageCount):
            if ImageType_Depth == stFrameData.stImageData[i].enImageType:
                stPointCloudImage = MV3D_RGBD_IMAGE_DATA()
                # ch:深度图转点云 | en:Depth map to point cloud
                ret =
camera.MV3D_RGBD_MapDepthToPointCloud(pointer(stFrameData.stImageData[i]),
pointer(stPointCloudImage))
                if MV3D_RGBD_OK != ret:
                    print("_MapDepthToPointCloud() Run failed...")
                else:
                    print("_MapDepthToPointCloud() Run Succeed: framenum (%d) height(%d)
width(%d)  len (%d)!" %( stPointCloudImage.nFrameNum,
                        stPointCloudImage.nHeight, stPointCloudImage.nWidth,
stPointCloudImage.nDataLen))
                    strMode = string_at(stPointCloudImage.pData, stPointCloudImage.nDataLen)
                    sValue = struct.unpack('f'*int(stPointCloudImage.nHeight *
stPointCloudImage.nWidth * 3), strMode)
                    with open("POINT3D_data.txt","w") as f:
                        for j in range(0, stPointCloudImage.nHeight *
stPointCloudImage.nWidth):
                            strPOINT3DValue = "X:{:f},Y:{:f},Z:{:f} \n".format(sValue[j *
3],sValue[j * 3 + 1],sValue[j * 3 + 2])
                            f.write(strPOINT3DValue)
    else:
        print("MV3D_RGBD_FetchFrame lost frame!")

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    print ("Main done")
    sys.exit()
```

6.16 SaveImage

该示例程序说明如何保存获取到的深度图、彩色图和原始图。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, MV3D_RGBD_OK, \
    FileType_BMP,ImageType_Depth, ImageType_RGB8_Planar,
ImageType_YUV420SP_NV12  ,ImageType_YUV420SP_NV21 , ImageType_YUV422, ImageType_Mono8

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
```

```
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    stFrameData = MV3D_RGBD_FRAME_DATA()
    # ch:获取图像数据 | en:Get image data
    ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
    if MV3D_RGBD_OK == ret:
        print("MV3D_RGBD_FetchFrame success.")
        for i in range(0, stFrameData.nImageCount):
            # ch:解析深度图像并保存 | en:Parse and save depth image
            if ImageType_Depth == stFrameData.stImageData[i].enImageType:
                print("Get depth image succeed: framenum (%d) height(%d) width(%d)
len(%d)!" %( stFrameData.stImageData[i].nFrameNum,
                    stFrameData.stImageData[i].nHeight, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nDataLen))
                chDepthFileName ="[{:d}]_Depth".format(stFrameData.stImageData[i].nFrameNum)
                ret = camera.MV3D_RGBD_SaveImage(pointer(stFrameData.stImageData[i]),
FileType_BMP, chDepthFileName)
                if MV3D_RGBD_OK == ret:
                    print("Save depth image success.")
                else:
                    print("Save depth image failed.")
            # ch:解析彩色图像并保存 | en:Parse and save color image
            elif ImageType_RGB8_Planar == stFrameData.stImageData[i].enImageType or
ImageType_YUV420SP_NV12 == stFrameData.stImageData[i].enImageType or
ImageType_YUV420SP_NV21 == stFrameData.stImageData[i].enImageType or ImageType_YUV422 ==
stFrameData.stImageData[i].enImageType:
                print("Get color image succeed: framenum (%d) height(%d) width(%d)
len(%d)!" % (stFrameData.stImageData[i].nFrameNum,
                    stFrameData.stImageData[i].nHeight, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nDataLen))
                chColorFileName =
"[{:d}]_Color".format(stFrameData.stImageData[i].nFrameNum)
                ret = camera.MV3D_RGBD_SaveImage(pointer(stFrameData.stImageData[i]),
FileType_BMP, chColorFileName)
                if MV3D_RGBD_OK == ret:
                    print("Save color image success.")
                else:
                    print("Save color image failed.")

            # ch:解析原始图像并保存 | en:Parse and save mono image
            elif ImageType_Mono8 == stFrameData.stImageData[i].enImageType:
                print("Get mono image succeed: framenum (%d) height(%d) width(%d)
len(%d)!" % (stFrameData.stImageData[i].nFrameNum,
                    stFrameData.stImageData[i].nHeight, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nDataLen))
                chMonoFileName = "[{:d}]_Mono".format(stFrameData.stImageData[i].nFrameNum)
                ret = camera.MV3D_RGBD_SaveImage(pointer(stFrameData.stImageData[i]),
FileType_BMP, chMonoFileName)
                if MV3D_RGBD_OK == ret:
                    print("Save mono image success.")
                else:
                    print("Save mono image failed.")
            # ch:解析其他类型图像 | en:Parse other types of images
```

```
            else:
                print("Get image succeed: framenum (%d) image type(%ld) height(%d) width(%d)
len(%d)!" %(stFrameData.stImageData[i].nFrameNum, stFrameData.stImageData[i].enImageType,
                    stFrameData.stImageData[i].nHeight, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nDataLen))
    else:
        print("MV3D_RGBD_FetchFrame lost frame!")

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    print ("Main done")
    sys.exit()
```

6.17 SaveTexturedPointCloudImage

该示例程序说明如何将获取到的点云数据和彩色图保存为纹理点云图像。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, MV3D_RGBD_OK,ImageType_Depth, \
    PointCloudType_Common,PointCloudType_Normals ,PointCloudType_Texture,
PointCloudType_Texture_Normals, MV3D_RGBD_ENUM_POINT_CLOUD_OUTPUT,PointCloudFileType_PLY,
\
    ImageType_PointCloud, ImageType_PointCloudWithNormals, ImageType_TexturedPointCloud,
ImageType_TexturedPointCloudWithNormals, ImageType_RGB8_Planar,ImageType_YUV422,
PointCloudFileType_PCD_ASCII, PointCloudFileType_PLY

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
```

```
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:设置点云输出节点 | en:Set point cloud output node
    stParam = MV3D_RGBD_PARAM()
    stParam.enParamType = ParamType_Enum
    stParam.ParamInfo.stEnumParam.nCurValue = PointCloudType_Common
    ret = camera.MV3D_RGBD_SetParam(MV3D_RGBD_ENUM_POINT_CLOUD_OUTPUT, pointer(stParam))

    if MV3D_RGBD_OK != ret:
        print ("SetParam fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    print ("Set point cloud output success.")

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    stFrameData = MV3D_RGBD_FRAME_DATA()
    # ch:获取图像数据 | en:Get image data
    ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
    if MV3D_RGBD_OK == ret:
        print("MV3D_RGBD_FetchFrame success.")
        pstPointCloudImage = MV3D_RGBD_IMAGE_DATA()
        pstTexture = MV3D_RGBD_IMAGE_DATA()
        for i in range(0, stFrameData.nImageCount):
            # ch:解析点云数据 | en: Parse point cloud data
            if ImageType_PointCloud == stFrameData.stImageData[i].enImageType:
                print("Get point cloud succeed: framenum (%d) height(%d) width(%d)
len(%d)!" %(stFrameData.stImageData[i].nFrameNum,
                        stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, stFrameData.stImageData[i].nDataLen))
                pstPointCloudImage = stFrameData.stImageData[i]
            if ImageType_RGB8_Planar == stFrameData.stImageData[i].enImageType:
                print("Get rgb8 planar succeed: framenum (%d) height(%d) width(%d)
len(%d)!" %(stFrameData.stImageData[i].nFrameNum,
                        stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, stFrameData.stImageData[i].nDataLen))
                pstTexture = stFrameData.stImageData[i]
        # ch:保存纹理点云图像 | en: Save textured point cloud image
        if ImageType_PointCloud == pstPointCloudImage.enImageType and
ImageType_RGB8_Planar == pstTexture.enImageType:
            chFileName =
"[{:d}]_TexturedPointCloudImage".format(pstPointCloudImage.nFrameNum)
            ret = camera.MV3D_RGBD_SaveTexturedPointCloudImage(pointer(pstPointCloudImage),
pointer(pstTexture), PointCloudFileType_PLY, chFileName)
            if MV3D_RGBD_OK == ret:
                print("Save %s success." % chFileName)
    else:
        print("MV3D_RGBD_FetchFrame lost frame!")
```

```
# ch:停止取流 | en:Stop grabbing
ret=camera.MV3D_RGBD_Stop()
if ret != 0:
    print ("stop fail! ret[0x%x]" % ret)
    os.system('pause')
    sys.exit()

# ch:销毁句柄 | en:Destroy the device handle
ret=camera.MV3D_RGBD_CloseDevice()
if ret != 0:
    print ("CloseDevice fail! ret[0x%x]" % ret)
    os.system('pause')
    sys.exit()

sys.exit()
```

6.18 TriggerMode

该示例程序说明如何通过软触发方式获取数据信息。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT

g_bExit = False
def work_thread(camera=0,pdata=0,nDataSize=0):
    while True:
        ret=camera.MV3D_RGBD_SoftTrigger()
        if ret==0:
            print("MV3D_RGBD_SoftTrigger success!")
        stFrameData=MV3D_RGBD_FRAME_DATA()
        ret=camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
        if ret==0:
            for i in range(0, stFrameData.nImageCount):

print("MV3D_RGBD_FetchFrame[%d]:nFrameNum[%d],nDataLen[%d],nWidth[%d],nHeight[%d]" % (
              i, stFrameData.stImageData[i].nFrameNum,
stFrameData.stImageData[i].nDataLen, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nHeight))
            time.sleep(3)
        else:
            print("no data[0x%x]" % ret)
        if g_bExit == True:
            break

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
```

```
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:设置触发模式 | en:Set trigger mode
    ret=camera.MV3D_RGBD_SoftTrigger()
    if ret != 0:
        print ("set softtrigger mode failed! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    # ch:创建软触发线程 | en: Create softTrigger thread
    try:
        hthreadhandle=threading.Thread(target=work_thread,args=(camera,None,None))
        hthreadhandle.start()
    except:
        print("error: unable to start thread")

    os.system('pause')
    g_bExit = True

    hthreadhandle.join()

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
```

6.19 GetCameraParam

该示例程序说明如何获取相机参数。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, MV3D_RGBD_OK,ImageType_Depth, \
        MV3D_RGBD_CAMERA_PARAM,MV3D_RGBD_INT_IMAGEALIGN

def PrintCameraParam(stCameraParam):
    print("-------- Depth Calib Info --------")
    print("Depth Width: %d" % stCameraParam.stDepthCalibInfo.nWidth)
    print("Depth Height: %d" % stCameraParam.stDepthCalibInfo.nHeight)
    print("--------")
    for i in range(0,9):
        print("Depth Intrinsic[%d]: %.10f" % (i,
stCameraParam.stDepthCalibInfo.stIntrinsic.fData[i]))
    print("--------")
    for i in range(0,12):
        print("Depth Distortion[%d]: %.10f" % ( i,
stCameraParam.stDepthCalibInfo.stDistortion.fData[i]))

    print("-------- Rgb Calib Info --------")
    print("Rgb Width: %d" % stCameraParam.stRgbCalibInfo.nWidth)
    print("Rgb Height: %d" % stCameraParam.stRgbCalibInfo.nHeight)
    print("--------")
    for i in range(0,9):
        print("Rgb Intrinsic[%d]: %.10f" % (i,
stCameraParam.stRgbCalibInfo.stIntrinsic.fData[i]))
    print("--------")
    for i in range(0,12):
        print("Rgb Distortion[%d]: %.10f" % (i,
stCameraParam.stRgbCalibInfo.stDistortion.fData[i]))
    print("-------- Depth to Rgb Extrinsic --------")
    for i in range(0,16):
        print("Depth to Rgb Extrinsic[%d]: %.10f" % (i,
stCameraParam.stDepth2RgbExtrinsic.fData[i]))
    print("--------")
    return

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
```

```
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:获取传感器标定参数 | en:Get sensor calib param
    stCameraParam = MV3D_RGBD_CAMERA_PARAM()
    ret = camera.MV3D_RGBD_GetCameraParam(pointer(stCameraParam))
    if MV3D_RGBD_OK != ret:
        print ("MV3D_RGBD_GetCameraParam fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()
    print("Get camera param success.")
    PrintCameraParam(stCameraParam)

    # ch:按任意键退出 | en:Press any key to quit
    print("Press any key to quit.")
    os.system('pause')

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    sys.exit()
```

第7章 第三方对接 Demo

Open3D 是一个基于 C++编写的开源库，它能提供丰富的 3D 数据处理和可视化功能，例如点云处理、网格处理、体积渲染、相机标定等。通过 Open3D，您可以轻松地读取、处理、渲染和保存 3D 数据，实现高效的 3D 应用开发。

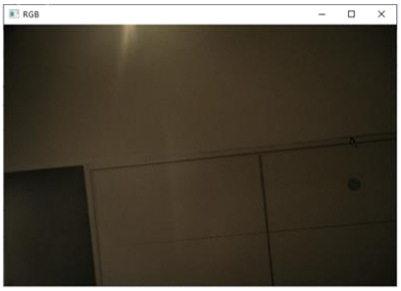
使用示例前，需准备 0.15.1 及以上版本的 Open3D 环境。

说明

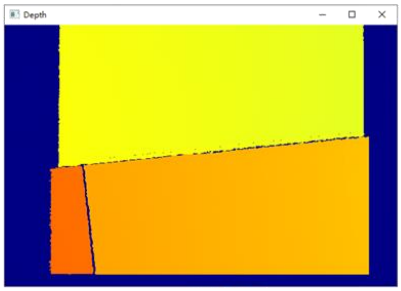
运行 ImageRead 前，需先执行 GetPointCloudImage 和 SaveImage，将保存的图像拷贝的程序运行目录下。

部分示例程序的运行效果如下图所示，供参考。

FetchFrame



彩色图

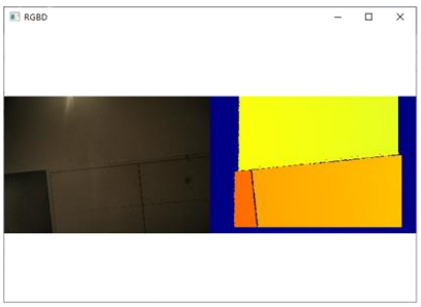


深度图

GetPointCloudImage

点云图

RecorderRGBD



RGB-D图

RGB-D图转换为点云图

7.1 FetchFrame

该示例程序调用 Open3D 接口对从 SDK 获取到的深度图和彩色图进行渲染。

```
# -- coding: utf-8 --
import threading
import ctypes
import time
import os
import open3d as o3d
import numpy as np;
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, ImageType_Depth,
ImageType_RGB8_Planar,ImageType_YUV422,ImageType_YUV420SP_NV12,ImageType_YUV420SP_NV21

g_bExit = False
def work_thread(camera=0,nDataSize=0):
    vis = o3d.visualization.Visualizer()
    vis.create_window(window_name='RGB', width=600, height=400)
    vis1 = o3d.visualization.Visualizer()
    vis1.create_window(window_name='DEPTH', width=600, height=400)
    imageDepth = o3d.geometry.Image()
    imageRgb = o3d.geometry.Image()
    while True:
        stFrameData=MV3D_RGBD_FRAME_DATA()
        ret=camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
        if ret==0:
            vis.clear_geometries()
            vis1.clear_geometries()
            for i in range(0, stFrameData.nImageCount):

print("MV3D_RGBD_FetchFrame[%d]:nFrameNum[%d],nDataLen[%d],nWidth[%d],nHeight[%d]" % (
                i, stFrameData.stImageData[i].nFrameNum,
stFrameData.stImageData[i].nDataLen, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nHeight))
                if ImageType_Depth == stFrameData.stImageData[i].enImageType:
                    strMode = string_at(stFrameData.stImageData[i].pData,
stFrameData.stImageData[i].nDataLen)
                    image = np.frombuffer(strMode, dtype=np.int16)
                    image = image.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                    imageDepth = o3d.geometry.Image(image)

                if ImageType_RGB8_Planar == stFrameData.stImageData[i].enImageType:
                    strMode = string_at(stFrameData.stImageData[i].pData,
stFrameData.stImageData[i].nDataLen)
                    image = np.frombuffer(strMode, dtype=np.uint8)
                    image = image.reshape(3, stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth)
                    image = image.T.reshape(stFrameData.stImageData[i].nDataLen)
                    image = image.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, 3)
                    imageRgb = o3d.geometry.Image(image)

                if ImageType_YUV422 == stFrameData.stImageData[i].enImageType:
                    YUV = string_at(stFrameData.stImageData[i].pData,
stFrameData.stImageData[i].nDataLen)
                    YUV_np = np.frombuffer(YUV, dtype=np.uint8)
                    YUV_np = YUV_np.reshape((int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 2), 4)
```

```
                YUV_np = YUV_np.T.reshape(stFrameData.stImageData[i].nDataLen)
                Y1 = YUV_np[0:(int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth /2)]
                U = YUV_np[(int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth/2) : stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth]
                U = np.repeat(U,2)
                Y2 = YUV_np[stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth : (int)(3 * stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth/2)]
                V = YUV_np[(int)(3 * stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 2) : 2 * stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth]
                V = np.repeat(V,2)

                Y = np.append(Y1, Y2)
                Y = Y.reshape(2, (int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth/2)).T
                Y = Y.reshape(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth)

                R = Y + 1.370705 * ( V - np.array([128]))
                G = Y - 0.698001 * ( U - np.array([128])) - (0.703125 * (V -
np.array([128])))
                B = Y + 1.732446 * ( U - np.array([128]))

                R = np.where(R < 0, 0, R)
                R = np.where(R > 255,255,R)
                G = np.where(G < 0, 0, G)
                G = np.where(G > 255,255,G)
                B = np.where(B < 0, 0, B)
                B = np.where(B > 255,255,B)

                RGB = np.zeros((stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, 3), dtype=np.uint8)
                RGB[:, :, 2] = B.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                RGB[:, :, 1] = G.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                RGB[:, :, 0] = R.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                imageRgb = o3d.geometry.Image(RGB)

            if ImageType_YUV420SP_NV12 == stFrameData.stImageData[i].enImageType:
                YUV = string_at(stFrameData.stImageData[i].pData,
stFrameData.stImageData[i].nDataLen)
                YUV_np = np.frombuffer(YUV, dtype=np.uint8)
                Y = YUV_np[0:stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth]
                UV = YUV_np[stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth:(int)(3 * stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 2)]
                UV = UV.reshape((int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 4), 2)
                UV = UV.T.reshape((int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 2))
                U = UV[0 : (int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 4)]
                U =
np.repeat(U,2).reshape((int)(stFrameData.stImageData[i].nHeight/2),stFrameData.stImageDat
a[i].nWidth)
                U = np.repeat(U,2, axis = 0).reshape(stFrameData.stImageData[i].nHeight
* stFrameData.stImageData[i].nWidth)

                V = UV[(int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 4) : (int)(stFrameData.stImageData[i].nHeight *
```

```
stFrameData.stImageData[i].nWidth / 2)]
                V =
np.repeat(V,2).reshape((int)(stFrameData.stImageData[i].nHeight/2),stFrameData.stImageDat
a[i].nWidth)
                V = np.repeat(V, 2, axis = 0).reshape(stFrameData.stImageData[i].nHeight
* stFrameData.stImageData[i].nWidth)

                R = Y + (140 * ( V- np.array([128])) ) / 100
                G = Y - (34 * (U - np.array([128]))) / 100 - (71 * (V -
np.array([128]))) / 100
                B = Y + (177 * (U - np.array([128]))) / 100

                R = np.where(R < 0, 0, R)
                R = np.where(R > 255,255,R)
                G = np.where(G < 0, 0, G)
                G = np.where(G > 255,255,G)
                B = np.where(B < 0, 0, B)
                B = np.where(B > 255,255,B)

                RGB = np.zeros((stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, 3), dtype=np.uint8)
                RGB[:, :, 2] = B.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                RGB[:, :, 1] = G.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                RGB[:, :, 0] = R.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                imageRgb = o3d.geometry.Image(RGB)

            if ImageType_YUV420SP_NV21 == stFrameData.stImageData[i].enImageType:
                YUV = string_at(stFrameData.stImageData[i].pData,
stFrameData.stImageData[i].nDataLen)
                YUV_np = np.frombuffer(YUV, dtype=np.uint8)
                Y = YUV_np[0:stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth]
                UV = YUV_np[stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth:(int)(3 * stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 2)]
                UV = UV.reshape((int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 4), 2)
                UV = UV.T.reshape((int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 2))
                V = UV[0 : (int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 4)]
                V =
np.repeat(V,2).reshape((int)(stFrameData.stImageData[i].nHeight/2),stFrameData.stImageDat
a[i].nWidth)
                V = np.repeat(V,2, axis = 0).reshape(stFrameData.stImageData[i].nHeight
* stFrameData.stImageData[i].nWidth)

                U = UV[(int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 4) : (int)(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth / 2)]
                U =
np.repeat(U,2).reshape((int)(stFrameData.stImageData[i].nHeight/2),stFrameData.stImageDat
a[i].nWidth)
                U = np.repeat(U, 2, axis = 0).reshape(stFrameData.stImageData[i].nHeight
* stFrameData.stImageData[i].nWidth)

                R = Y + (140 * ( V- np.array([128])) ) / 100
                G = Y - (34 * (U - np.array([128]))) / 100 - (71 * (V -
np.array([128]))) / 100
                B = Y + (177 * (U - np.array([128]))) / 100

                R = np.where(R < 0, 0, R)
                R = np.where(R > 255,255,R)
```

```
                G = np.where(G < 0, 0, G)
                G = np.where(G > 255,255,G)
                B = np.where(B < 0, 0, B)
                B = np.where(B > 255,255,B)

                RGB = np.zeros((stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, 3), dtype=np.uint8)
                RGB[:, :, 2] = B.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                RGB[:, :, 1] = G.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                RGB[:, :, 0] = R.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                imageRgb = o3d.geometry.Image(RGB)

          vis.add_geometry(imageRgb)
          vis.poll_events()
          vis.update_renderer()

          vis1.add_geometry(imageDepth)
          vis1.poll_events()
          vis1.update_renderer()
       else:
          print("no data[0x%x]" % ret)
       if g_bExit == True:
          break
   vis.destroy_window()
   vis1.destroy_window()

if __name__ == "__main__":
   nDeviceNum=ctypes.c_uint(0)
   nDeviceNum_p=byref(nDeviceNum)
   # ch:获取设备数量 | en:Get device number
   ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB ,
nDeviceNum_p)
   if  ret!=0:
      print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
      os.system('pause')
      sys.exit()
   if  nDeviceNum==0:
      print("find no device!")
      os.system('pause')
      sys.exit()
   print("Find devices numbers:", nDeviceNum.value)

   stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
   net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB ,
pointer(stDeviceList.DeviceInfo[0]), 20, nDeviceNum_p)

   for i in range(0, nDeviceNum.value):
      print("\ndevice: [%d]" % i)
      strModeName = ""
      for per in stDeviceList.DeviceInfo[i].chModelName:
         strModeName = strModeName + chr(per)
      print("device model name: %s" % strModeName)

      strSerialNumber = ""
      for per in stDeviceList.DeviceInfo[i].chSerialNumber:
         strSerialNumber = strSerialNumber + chr(per)
      print("device SerialNumber: %s" % strSerialNumber)

   # ch:创建相机示例 | en:Create a camera instance
   camera=Mv3dRgbd()
   nConnectionNum = input("please input the number of the device to connect:")
   if int(nConnectionNum) >= nDeviceNum.value:
```

```
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    # ch:获取图像线程 | en:Get image thread
    try:
        hthreadhandle=threading.Thread(target=work_thread,args=(camera,None))
        hthreadhandle.start()
    except:
        print("error: unable to start thread")

    time.sleep(20)
    g_bExit = True
    hthreadhandle.join()

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:销毁句柄 | en:Destroy the device handle
    ret=camera.MV3D_RGBD_CloseDevice()
    if ret != 0:
        print ("CloseDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    sys.exit()
```

7.2 GetPointCloudImage

该示例程序调用 Open3D 接口对从 SDK 获取的点云图及 RGB 数据转换成 Open3D 的点云格式，后调用 Open3D 接口进行渲染及保存。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
import open3d as o3d;
import numpy as np;
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, MV3D_RGBD_OK,ImageType_Depth, \

PointCloudType_Common,PointCloudType_Normals ,PointCloudType_Texture,PointCloudType_Textu
re_Normals,MV3D_RGBD_ENUM_POINT_CLOUD_OUTPUT, \
        ImageType_PointCloud, ImageType_PointCloudWithNormals,
ImageType_TexturedPointCloud, ImageType_TexturedPointCloudWithNormals,
PointCloudFileType_PLY,ImageType_RGB8_Planar

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
```

```
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()

    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:设置点云输出节点 | en:Set point cloud output node
    stParam = MV3D_RGBD_PARAM()
    stParam.enParamType = ParamType_Enum
    stParam.ParamInfo.stEnumParam.nCurValue = PointCloudType_Common
    ret = camera.MV3D_RGBD_SetParam(MV3D_RGBD_ENUM_POINT_CLOUD_OUTPUT, pointer(stParam))

    if MV3D_RGBD_OK != ret:
        print ("SetParam fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    print ("Set point cloud output success.")

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    stFrameData = MV3D_RGBD_FRAME_DATA()
    # ch:获取图像数据 | en:Get image data
    ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
    if MV3D_RGBD_OK == ret:
        print("MV3D_RGBD_FetchFrame success.")
        pcd = o3d.geometry.PointCloud()
        for i in range(0, stFrameData.nImageCount):
            # ch:解析点云数据 | en:Parse point cloud data
            if ImageType_PointCloud == stFrameData.stImageData[i].enImageType:
                print("Get point cloud succeed: framenum (%d) height(%d) width(%d)
len(%d)!" %(stFrameData.stImageData[i].nFrameNum,
                        stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, stFrameData.stImageData[i].nDataLen))
                strMode = string_at(stFrameData.stImageData[i].pData,
stFrameData.stImageData[i].nDataLen)
                xyz = np.frombuffer(strMode, dtype=np.float32)
                xyz = xyz.reshape(stFrameData.stImageData[i].nWidth *
stFrameData.stImageData[i].nHeight, 3)
                pcd.points = o3d.utility.Vector3dVector(xyz)


            if ImageType_RGB8_Planar == stFrameData.stImageData[i].enImageType:
                print("Get rgb8 planar succeed: framenum (%d) height(%d) width(%d)
len(%d)!" %(stFrameData.stImageData[i].nFrameNum,
                        stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, stFrameData.stImageData[i].nDataLen))
                strMode = string_at(stFrameData.stImageData[i].pData,
stFrameData.stImageData[i].nDataLen)
                rgb = np.frombuffer(strMode, dtype=np.uint8)
                rgb = rgb.reshape(3, stFrameData.stImageData[i].nHeight *
```

```
stFrameData.stImageData[i].nWidth)
             rgb = rgb.T.reshape(stFrameData.stImageData[i].nDataLen)
             rgb = rgb.reshape(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth, 3)
             rgbColor = np.multiply(rgb, 1/255)
             pcd.colors=o3d.utility.Vector3dVector(rgbColor)

       o3d.io.write_point_cloud("PointCloud.ply", pcd, write_ascii=True)
       o3d.visualization.draw_geometries([pcd],window_name="Open3d点云渲染
",width=600,height=450)

   else:
       print("MV3D_RGBD_FetchFrame lost frame!")

   # ch:停止取流 | en:Stop grabbing
   ret=camera.MV3D_RGBD_Stop()
   if ret != 0:
       print ("stop fail! ret[0x%x]" % ret)
       os.system('pause')
       sys.exit()

   # ch:销毁句柄 | en:Destroy the device handle
   ret=camera.MV3D_RGBD_CloseDevice()
   if ret != 0:
       print ("CloseDevice fail! ret[0x%x]" % ret)
       os.system('pause')
       sys.exit()

   sys.exit()
```

7.3 ImageRead

该示例程序调用 Open3D 接口对从本地读取深度图、彩色图转换成 RGBD 图像并渲染，读取点云图像转换成 Open3D 格式并渲染。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
import open3d as o3d;
import numpy as np;

if __name__ == "__main__":

    rgbimage = o3d.io.read_image("Color.png")
    depthImage = o3d.io.read_image("Depth.png")
    Open3dRgbd = o3d.geometry.RGBDImage.create_from_color_and_depth(rgbimage, depthImage,
depth_scale = 1000, convert_rgb_to_intensity = False)
    o3d.visualization.draw_geometries([Open3dRgbd],window_name ="Open3d RGBD图像渲染
",width=600,height=450)

    pointCloud = o3d.io.read_point_cloud("PointCloud.ply")
    o3d.visualization.draw_geometries([pointCloud],window_name ="Open3d点云渲染
",width=600,height=450)
```

7.4 RecorderRGBD

该示例程序调用 Open3D 接口对从 SDK 获取的深度图和彩色图转换成 RGBD 图像，调用 Open3D 接口将 RGBD 图像转换成点云图像并进行渲染。

```
# -- coding: utf-8 --
import threading
import ctypes
import time
import os
import open3d as o3d
import numpy as np;
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT,
MV3D_RGBD_INT_OUTPUT_RGBD,ImageType_Rgbd,MV3D_RGBD_CAMERA_PARAM,MV3D_RGBD_FLOAT_Z_UNIT

g_bExit = False
def work_thread(camera=0, rgbIntrinsic=0 , ZUnit= 0, nDataSize=0):
    vis = o3d.visualization.Visualizer()
    vis.create_window(window_name='RGBD', width=600, height=400)

    vis1 = o3d.visualization.Visualizer()
    vis1.create_window(window_name='RGBD-To-PointCloud', width=600, height=400)
    while True:
        stFrameData=MV3D_RGBD_FRAME_DATA()
        ret=camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
        if ret==0:
            vis.clear_geometries()
            vis1.clear_geometries()
            for i in range(0, stFrameData.nImageCount):

print("MV3D_RGBD_FetchFrame[%d]:nFrameNum[%d],nDataLen[%d],nWidth[%d],nHeight[%d]" % (
                i, stFrameData.stImageData[i].nFrameNum,
stFrameData.stImageData[i].nDataLen, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nHeight))
                if ImageType_Rgbd == stFrameData.stImageData[i].enImageType:
                    strMode = string_at(stFrameData.stImageData[i].pData,
stFrameData.stImageData[i].nDataLen)
                    imageRgbd = np.frombuffer(strMode, dtype=np.int8)

                    imageRgbd = imageRgbd.reshape(stFrameData.stImageData[i].nHeight ,
stFrameData.stImageData[i].nWidth , 5)
                    RGB = np.zeros((stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, 3), dtype=np.uint8)
                    RGB[:, :, 2] = imageRgbd[:,:,2]
                    RGB[:, :, 1] = imageRgbd[:,:,1]
                    RGB[:, :, 0] = imageRgbd[:,:,0]
                    Open3dRgb = o3d.geometry.Image(RGB)

                    D1 = imageRgbd[:,:,3].reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                    D2 = imageRgbd[:,:,4].reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                    imageDepthUin8 = np.append(D1,D2)
                    imageDepthUin8 =
imageDepthUin8.reshape(2,stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth).T.reshape(stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth * 2)
                    depthBytesArr = imageDepthUin8.tobytes()
```

```
                Depth = np.frombuffer(depthBytesArr, dtype=np.int16)
                Depth = Depth.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)

                Open3dDepth = o3d.geometry.Image(Depth)

                Open3dRgbd =
o3d.geometry.RGBDImage.create_from_color_and_depth(Open3dRgb, Open3dDepth, depth_scale =
(1000 / ZUnit) , depth_trunc = 3, convert_rgb_to_intensity = False)
                vis.add_geometry(Open3dRgbd)
                vis.poll_events()
                vis.update_renderer()

                inter = o3d.camera.PinholeCameraIntrinsic()
                # W H fx fy cx cy
                inter.set_intrinsics(stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nHeight, rgbIntrinsic[0], rgbIntrinsic[4], rgbIntrinsic[2],
rgbIntrinsic[5])
                pcd = o3d.geometry.PointCloud.create_from_rgbd_image(Open3dRgbd, inter)

                vis1.add_geometry(pcd)
                vis1.poll_events()
                vis1.update_renderer()
      else:
         print("no data[0x%x]" % ret)
      if g_bExit == True:
         break
   vis.destroy_window()

if __name__ == "__main__":
   nDeviceNum=ctypes.c_uint(0)
   nDeviceNum_p=byref(nDeviceNum)
   # ch:获取设备数量 | en:Get device number
   ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB ,
nDeviceNum_p)
   if  ret!=0:
      print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
      os.system('pause')
      sys.exit()
   if  nDeviceNum==0:
      print("find no device!")
      os.system('pause')
      sys.exit()
   print("Find devices numbers:", nDeviceNum.value)

   stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
   net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB ,
pointer(stDeviceList.DeviceInfo[0]), 20, nDeviceNum_p)

   for i in range(0, nDeviceNum.value):
      print("\ndevice: [%d]" % i)
      strModeName = ""
      for per in stDeviceList.DeviceInfo[i].chModelName:
         strModeName = strModeName + chr(per)
      print("device model name: %s" % strModeName)

      strSerialNumber = ""
      for per in stDeviceList.DeviceInfo[i].chSerialNumber:
         strSerialNumber = strSerialNumber + chr(per)
      print("device SerialNumber: %s" % strSerialNumber)

   # ch:创建相机示例 | en:Create a camera instance
   camera=Mv3dRgbd()
   nConnectionNum = input("please input the number of the device to connect:")
   if int(nConnectionNum) >= nDeviceNum.value:
      print("intput error!")
```

```
    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:设置RGBD输出节点 | en:Set RGBD image output node
    stParam = MV3D_RGBD_PARAM()
    stParam.enParamType = ParamType_Int
    stParam.ParamInfo.stIntParam.nCurValue = 1
    ret = camera.MV3D_RGBD_SetParam(MV3D_RGBD_INT_OUTPUT_RGBD, pointer(stParam))
    if ret != 0:
        print ("SetParam fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    stCameraParam = MV3D_RGBD_CAMERA_PARAM()
    ret = camera.MV3D_RGBD_GetCameraParam(pointer(stCameraParam))
    if ret != 0:
        print ("GetCameraParam fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    intrinsic = np.array(stCameraParam.stRgbCalibInfo.stIntrinsic.fData).reshape(9)

    ret = camera.MV3D_RGBD_GetParam(MV3D_RGBD_FLOAT_Z_UNIT, pointer(stParam))
    if ret != 0:
        print ("GetParam fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()
    ZUnit = stParam.ParamInfo.stFloatParam.fCurValue


    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    # ch:获取图像线程 | en:Get image thread
    try:
        hthreadhandle=threading.Thread(target=work_thread, args=(camera, intrinsic, ZUnit,
None))
        hthreadhandle.start()
    except:
        print("error: unable to start thread")

    time.sleep(20)
    g_bExit = True
    hthreadhandle.join()

    # ch:停止取流 | en:Stop grabbing
    ret=camera.MV3D_RGBD_Stop()
    if ret != 0:
        print ("stop fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
```

```
# ch:销毁句柄 | en:Destroy the device handle
ret=camera.MV3D_RGBD_CloseDevice()
if ret != 0:
    print ("CloseDevice fail! ret[0x%x]" % ret)
    os.system('pause')
    sys.exit()

sys.exit()
```

7.5 SaveImage

该示例程序调用 Open3D 接口对从 SDK 获取的深度图、彩色图及 Mono 图像并转换成 Open3D 格式,后调用 Open3D 接口进行图像保存。

```
# -- coding: utf-8 --
import threading
import msvcrt
import ctypes
import time
import os
import struct
import open3d as o3d
import numpy as np;
from ctypes import *
from Mv3dRgbdImport.Mv3dRgbdDefine import *
from Mv3dRgbdImport.Mv3dRgbdApi import *
from Mv3dRgbdImport.Mv3dRgbdDefine import DeviceType_Ethernet, DeviceType_USB,
DeviceType_Ethernet_Vir, DeviceType_USB_Vir, MV3D_RGBD_FLOAT_EXPOSURETIME, \
    ParamType_Float, ParamType_Int, ParamType_Enum, CoordinateType_Depth,
MV3D_RGBD_FLOAT_Z_UNIT, MV3D_RGBD_OK, \
    FileType_BMP,ImageType_Depth, ImageType_RGB8_Planar,
ImageType_YUV420SP_NV12  ,ImageType_YUV420SP_NV21 , ImageType_YUV422, ImageType_Mono8

if __name__ == "__main__":
    nDeviceNum=ctypes.c_uint(0)
    nDeviceNum_p=byref(nDeviceNum)
    # ch:获取设备数量 | en:Get device number
    ret=Mv3dRgbd.MV3D_RGBD_GetDeviceNumber(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, nDeviceNum_p)
    if  ret!=0:
        print("MV3D_RGBD_GetDeviceNumber fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()
    if  nDeviceNum==0:
        print("find no device!")
        os.system('pause')
        sys.exit()
    print("Find devices numbers:", nDeviceNum.value)

    stDeviceList = MV3D_RGBD_DEVICE_INFO_LIST()
    net = Mv3dRgbd.MV3D_RGBD_GetDeviceList(DeviceType_Ethernet | DeviceType_USB |
DeviceType_Ethernet_Vir | DeviceType_USB_Vir, pointer(stDeviceList.DeviceInfo[0]), 20,
nDeviceNum_p)

    for i in range(0, nDeviceNum.value):
        print("\ndevice: [%d]" % i)
        strModeName = ""
        for per in stDeviceList.DeviceInfo[i].chModelName:
            strModeName = strModeName + chr(per)
        print("device model name: %s" % strModeName)

        strSerialNumber = ""
        for per in stDeviceList.DeviceInfo[i].chSerialNumber:
            strSerialNumber = strSerialNumber + chr(per)
        print("device SerialNumber: %s" % strSerialNumber)

    # ch:创建相机示例 | en:Create a camera instance
    camera=Mv3dRgbd()
    nConnectionNum = input("please input the number of the device to connect:")
    if int(nConnectionNum) >= nDeviceNum.value:
        print("intput error!")
        os.system('pause')
        sys.exit()
```

```
    # ch:打开设备 | en:Open device
    ret =
camera.MV3D_RGBD_OpenDevice(pointer(stDeviceList.DeviceInfo[int(nConnectionNum)]))
    if ret != 0:
        print ("MV3D_RGBD_OpenDevice fail! ret[0x%x]" % ret)
        os.system('pause')
        sys.exit()

    # ch:开始取流 | en:Start grabbing
    ret=camera.MV3D_RGBD_Start()
    if ret != 0:
        print ("start fail! ret[0x%x]" % ret)
        camera.MV3D_RGBD_CloseDevice()
        os.system('pause')
        sys.exit()

    stFrameData = MV3D_RGBD_FRAME_DATA()
    # ch:获取图像数据 | en:Get image data
    ret = camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
    if MV3D_RGBD_OK == ret:
        print("MV3D_RGBD_FetchFrame success.")
        for i in range(0, stFrameData.nImageCount):
            # ch:解析深度图像并保存 | en:Parse and save depth image
            if ImageType_Depth == stFrameData.stImageData[i].enImageType:
                print("Get depth image succeed: framenum (%d) height(%d) width(%d)
len(%d)!" %( stFrameData.stImageData[i].nFrameNum,
                    stFrameData.stImageData[i].nHeight, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nDataLen))
                chDepthFileName ="Depth.png"
                imageDepth = o3d.geometry.Image()
                strMode = string_at(stFrameData.stImageData[i].pData,
stFrameData.stImageData[i].nDataLen)
                image = np.frombuffer(strMode, dtype=np.int16)
                image = image.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
                imageDepth = o3d.geometry.Image(image)
                o3d.io.write_image(chDepthFileName,imageDepth)
            # ch:解析彩色图像并保存 | en:Parse and save color image
            elif ImageType_RGB8_Planar == stFrameData.stImageData[i].enImageType:
                print("Get color image succeed: framenum (%d) height(%d) width(%d)
len(%d)!" % (stFrameData.stImageData[i].nFrameNum,
                    stFrameData.stImageData[i].nHeight, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nDataLen))
                chColorFileName = "Color.png"
                strMode = string_at(stFrameData.stImageData[i].pData,
stFrameData.stImageData[i].nDataLen)
                image = np.frombuffer(strMode, dtype=np.uint8)
                image = image.reshape(3, stFrameData.stImageData[i].nHeight *
stFrameData.stImageData[i].nWidth)
                image = image.T.reshape(stFrameData.stImageData[i].nDataLen)
                image = image.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth, 3)
                imageRgb = o3d.geometry.Image(image)
                o3d.io.write_image(chColorFileName,imageRgb)

            # ch:解析原始图像并保存 | en:Parse and save mono image
            elif ImageType_Mono8 == stFrameData.stImageData[i].enImageType:
                print("Get mono image succeed: framenum (%d) height(%d) width(%d)
len(%d)!" % (stFrameData.stImageData[i].nFrameNum,
                    stFrameData.stImageData[i].nHeight, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nDataLen))
                chMonoFileName = "Mono.png"
                imageMono = o3d.geometry.Image()
                strMode = string_at(stFrameData.stImageData[i].pData,
```

```
stFrameData.stImageData[i].nDataLen)
             image = np.frombuffer(strMode, dtype=np.uint8)
             image = image.reshape(stFrameData.stImageData[i].nHeight,
stFrameData.stImageData[i].nWidth)
             imageMono = o3d.geometry.Image(image)
             o3d.io.write_image(chMonoFileName,imageMono)

          # ch:解析其他类型图像 | en:Parse other types of images
          else:
             print("Get image succeed: framenum (%d) image type(%ld) height(%d) width(%d)
len(%d)!" %(stFrameData.stImageData[i].nFrameNum, stFrameData.stImageData[i].enImageType,
                 stFrameData.stImageData[i].nHeight, stFrameData.stImageData[i].nWidth,
stFrameData.stImageData[i].nDataLen))
   else:
      print("MV3D_RGBD_FetchFrame lost frame!")

   # ch:停止取流 | en:Stop grabbing
   ret=camera.MV3D_RGBD_Stop()
   if ret != 0:
      print ("stop fail! ret[0x%x]" % ret)
      os.system('pause')
      sys.exit()

   # ch:销毁句柄 | en:Destroy the device handle
   ret=camera.MV3D_RGBD_CloseDevice()
   if ret != 0:
      print ("CloseDevice fail! ret[0x%x]" % ret)
      os.system('pause')
      sys.exit()

   print ("Main done")
   sys.exit()
```

第8章 状态码定义

8.1 正确码定义

返回类型	类型值	类型信息描述
MV3D_RGBD_OK	0x00000000	成功，无错误

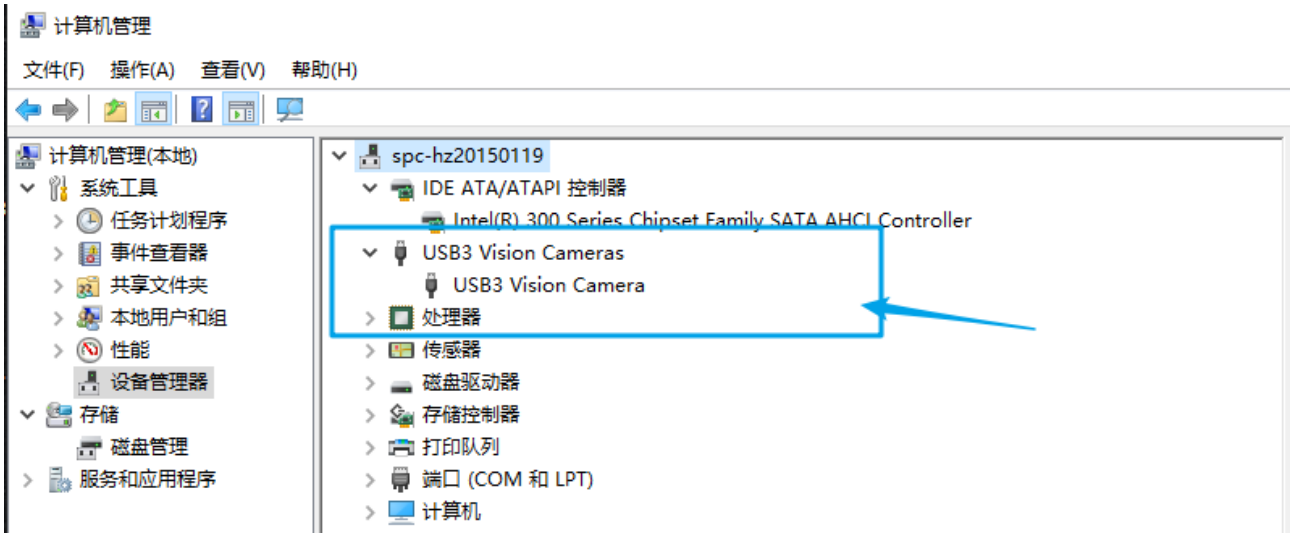
8.2 错误码定义

返回类型	类型值	类型信息描述
通用错误码定义：范围 0x80060000-0x800600FF		
MV3D_RGBD_E_HANDLE	0x80060000	错误或无效的句柄
MV3D_RGBD_E_SUPPORT	0x80060001	不支持的功能
MV3D_RGBD_E_BUFOVER	0x80060002	缓存已满
MV3D_RGBD_E_CALLORDER	0x80060003	函数调用顺序有误
MV3D_RGBD_E_PARAMETER	0x80060004	错误的参数
MV3D_RGBD_E_RESOURCE	0x80060005	资源申请失败
MV3D_RGBD_E_NODATA	0x80060006	无数据
MV3D_RGBD_E_PRECONDITION	0x80060007	前置条件有误，或运行环境已发生变化
MV3D_RGBD_E_VERSION	0x80060008	版本不匹配
MV3D_RGBD_E_NOENOUGH_BUF	0x80060009	传入的内存空间不足
MV3D_RGBD_E_ABNORMAL_IMAGE	0x8006000A	异常图像，可能是丢包导致图像不完整
MV3D_RGBD_E_LOAD_LIBRARY	0x8006000B	动态导入 DLL 失败
MV3D_RGBD_E_ALGORITHM	0x8006000C	算法错误
MV3D_RGBD_E_DEVICE_OFFLINE	0x8006000D	设备离线
MV3D_RGBD_E_ACCESS_DENIED	0x8006000E	设备无访问权限

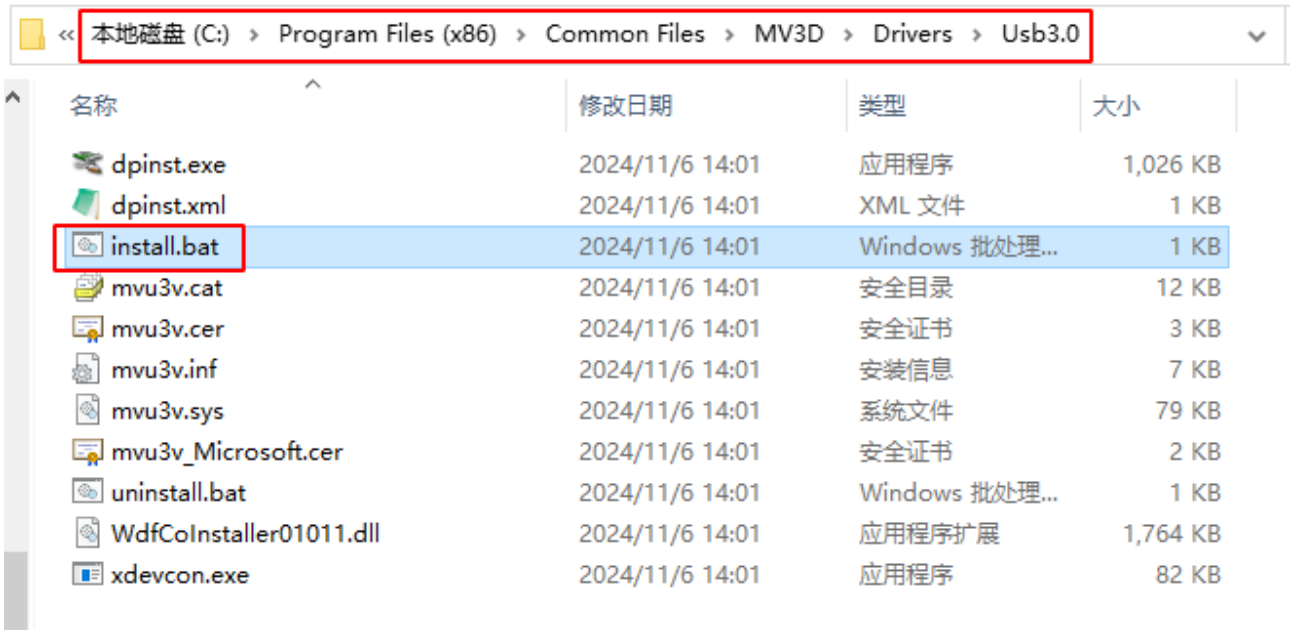
返回类型	类型值	类型信息描述
MV3D_RGBD_E_ACCESS_DENIED	0x8006000E	设备无访问权限
MV3D_RGBD_E_OUTOFRANGE	0x8006000F	值超出范围
MV3D_RGBD_E_UPG_FILE_MISMATCH	0x80060010	升级固件不匹配
MV3D_RGBD_E_UPG_CONFLICT	0x80060012	升级冲突
MV3D_RGBD_E_UPG_INNER_ERR	0x80060013	升级时相机内部出现错误
MV3D_RGBD_E_INDUSTRY	0x80060020	行业属性不匹配导致打开设备失败
MV3D_RGBD_E_NETWORK	0x80060021	网络相关错误
MV3D_RGBD_E_USB_WRITE	0x80060030	写 USB 出错
MV3D_RGBD_E_UNKNOW	0x800600FF	未知的错误

第9章 常见问题

问题 1： Windows 上枚举不到 USB 相机，如下图所示。



解决方法： 手动双击 install.bat 安装 USB 相机驱动，如下图所示。



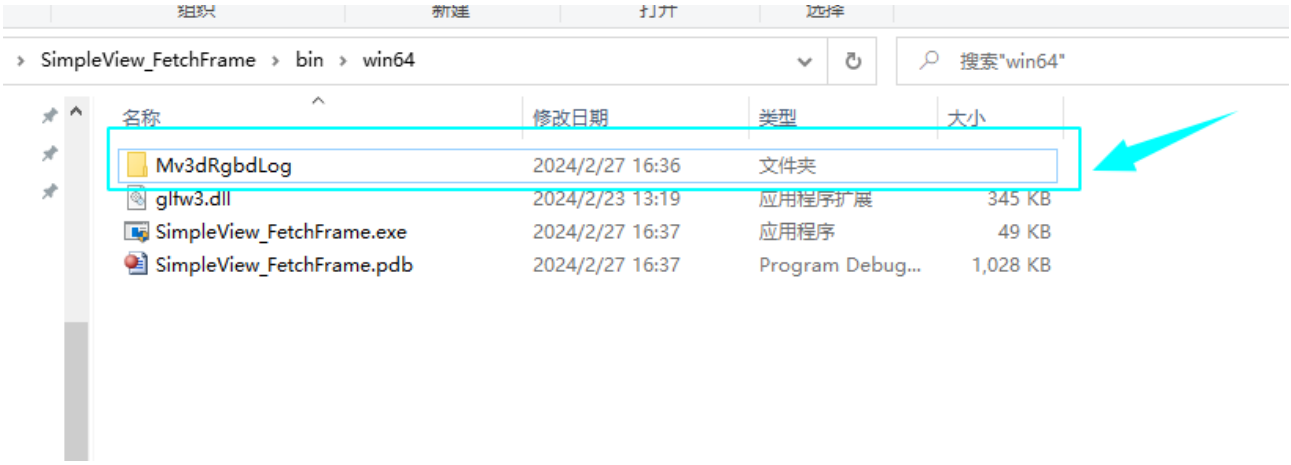
问题 2： 图像数据如何解析。

解决方法：在 BasicDemo.py 示例中有源码展示，如下图所示。

```
def work_thread(camera=0,pdata=0,nDataSize=0):
    while True:
        stFrameData=MV3D_RGBD_FRAME_DATA()
        ret=camera.MV3D_RGBD_FetchFrame(pointer(stFrameData), 5000)
        if ret==0:
            for i in range(0, stFrameData.nImageCount):
                print("MV3D_RGBD_FetchFrame[%d]:nFrameNum[%d],nDataLen[%d],nWidth[%d],nHeight[%d]" % (
                i, stFrameData.stImageData[i].nFrameNum, stFrameData.stImageData[i].nDataLen, stFrameData.stImageData[i].nWidth, stFrameData.stImageData[i].nHeight))
        else:
            print("no data[0x%x]" % ret)
```

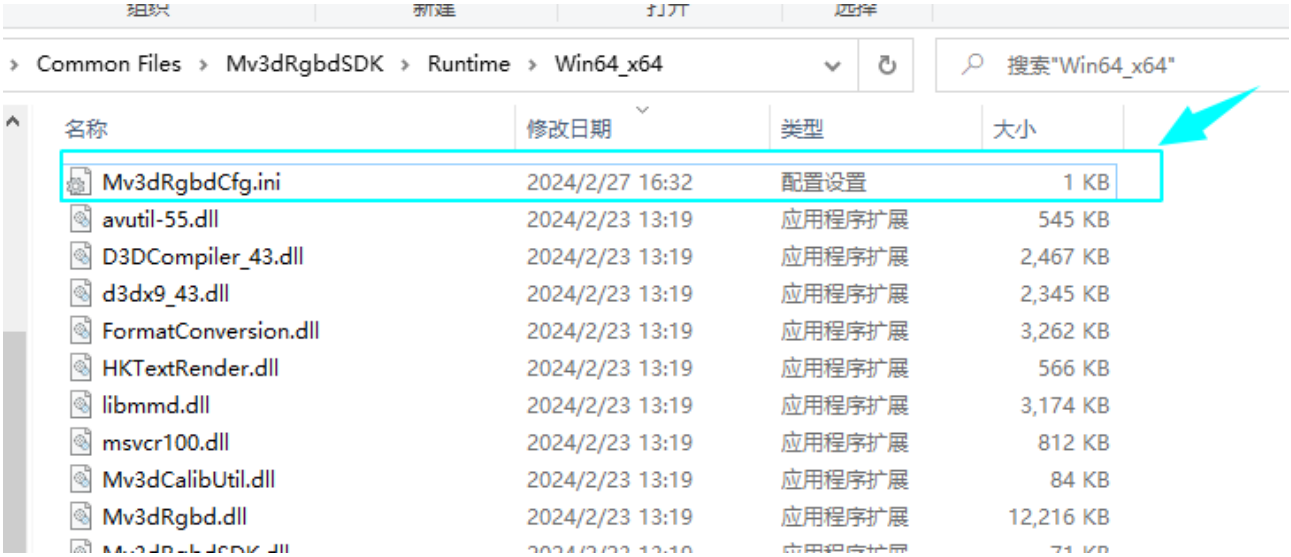
问题 3：无法找到日志文件。

解决方法：在可执行目录下找到 Mv3dRgbdLog 文件夹，里面有保存日志文件，如下图所示。



问题 4：如何获取更多的日志信息

解决方法：在 SDK 路径下找到 Mv3dRgbdCfg.ini 配置文件，按需求修改日志相关的配置参数，如下图所示。



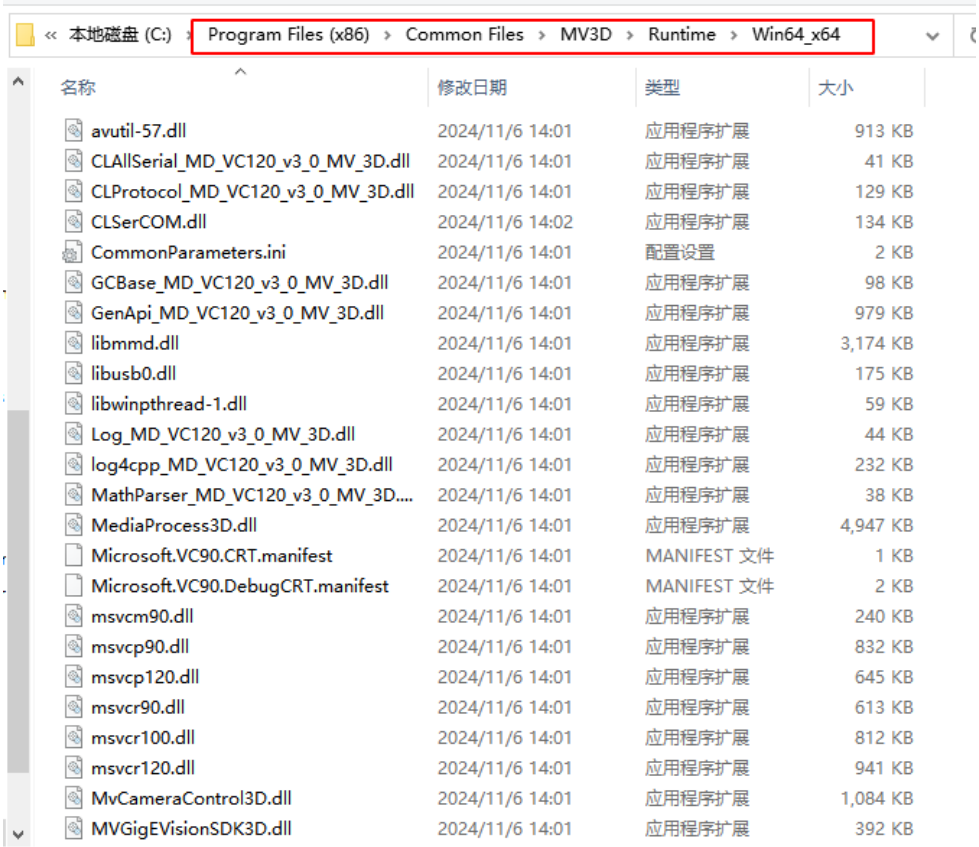
问题 5：USB 相机出图帧率异常。

解决方法：使用 USB2.0 线可能会导致该问题，检查 USB 接口是否为 USB3.0。

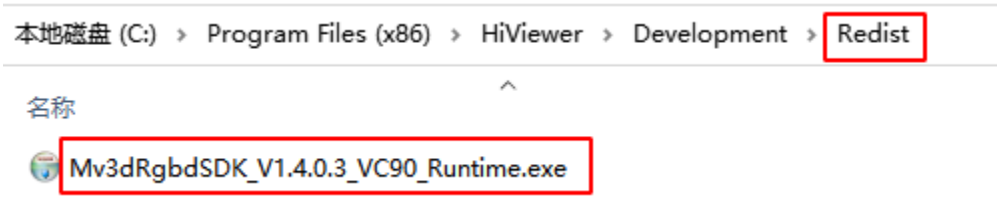
问题 6：枚举相机时，报错提示：0x8006000B 动态库导入失败

解决方法：

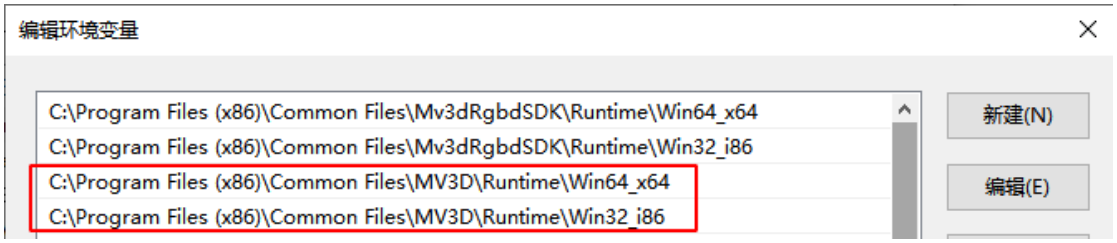
1、查看 C:\Program Files (x86)\Common Files\MV3D\Runtime 路径下 RGB-D SDK 依赖的工业 SDK 库是否被正确



2、若找不到相关库文件，则执行 Redist 文件夹下的安装包，重新安装 SDK。



3、若问题仍未解决，则查看系统环境变量中是否存在 MV3D SDK 库路径。若没有相关路径，则需手动配置，保存环境变量后再重新运行程序。



问题 7： Python3.8 及相关版本打开示例程序

解决方法： 在 Python\Mv3dRgbdImport 目录中打开 Mv3dRgbdApi.py 文件，将指定库路径的代码取消注释。

```
# ch: python3.8及相关版本可以指定如下路径的库 | en: python3.8 and related versions can specify libraries for the following paths
# os.add_dll_directory("C:\Program Files (x86)\Common Files\MV3D\Runtime\Win64_x64")
# os.add_dll_directory("C:\Program Files (x86)\Common Files\Mv3dRgbdSDK\Runtime\Win64_x64")
```